



USMAN INSTITUTE OF TECHNOLOGY

Affiliated with NED University of Engineering & Technology, Karachi

Department of Electrical Engineering

B.E. Electrical (Computer Systems Engineering)

FINAL YEAR PROJECT REPORT

Batch-2019

MULTICORE SOC FOR SMP LINUX

By

Abdul Muheet Ghani	(19B-012-EE)
Nameer Iqbal Ansari	(19B-023-CE)
Syeda Rafia Fizza Naveed	(19B-053-CE)
Khadija	(19B-060-CE)

Supervised by

Dr. Ali Ahmed Ansari, Assistant Professor
Engr. Asad Hussain, Senior Lecturer

DECLARATION

I hereby declare that this project report is based on my original work except for citations and quotations which have been duly acknowledged. I also declare that it has not been previously and concurrently submitted for any other degree or award at USMAN INSTITUTE OF TECHNOLOGY or other institutions.

Student Name (Student ID): Abdul Muheet Ghani (19B-012-EE)

Signature: _____

Student Name (Student ID): Nameer Iqbal Ansari (19B-023-CE)

Signature: _____

Student Name (Student ID): Syeda Rafia Fizza Naveed (19B-053-CE)

Signature: _____

Student Name (Student ID): Khadija (19B-060-CE)

Signature: _____

Date: _____

ACKNOWLEDGEMENT

We are thankful to the Micro Electronics Research Lab for providing us with the platform to work on the cutting edge technology. We are also thankful to Dr. Ali Ahmed, Engr. Sajjad Ahmed and Engr. Zeeshan Rafique for mentoring us throughout the project. Thanks to the whole faculty of the Usman Institute of Technology who evaluated our project to make it more worthy. Lastly, we are thankful to our sponsoring company XCELERIUM for funding this project, providing the resources, and giving us the initial hands-on experience from the project NOVA [1]. Since that project involved dealing with AWS-FPGA, this provided us with background knowledge and skills.

Thanks all for providing us with space and useful tools which helped us to complete this project in a successful way.

MULTICORE SOC FOR SMP LINUX

ABSTRACT

The project is situated within the domain of semiconductor technologies, exploring the fascinating interaction between hardware and software. The project utilized openpiton, a remarkable opensource framework renowned for generating multicore systems. This framework enables the creation of complex and efficient hardware, capable of supporting multiple cores and facilitating parallel processing.

To facilitate hardware simulation, the project also makes use of verilator, another opensource tool. It allows the developer to simulate and verify the functionality of the hardware designs, providing a crucial step in the development process before actual implementation.

From the software perspective, the project takes a significant stride by booting SMP (symmetric multiprocessing) linux over the hardware generated by openpiton. SMP linux is an advanced operating system that optimally utilizes multiple processor cores, ensuring smooth multitasking and enhanced performance.

By combining these opensource tools and technologies, the project offers an interesting journey from the foundational hardware level to the software abstractions that leverage the power of multicore systems. It serves as an exemplary illustration of the relationship between hardware and software, demonstrating how they collaborate to create a powerful computing environment. Moreover, through the use of opensource tools, the project promotes accessibility and encourages the wider community to explore and contribute to the fascinating world of semiconductor technologies.

TABLE OF CONTENTS

DECLARATION	I
ACKNOWLEDGEMENT	II
ABSTRACT	III
TABLE OF CONTENTS	IV
LIST OF FIGURES	V
LIST OF TABLES	VII
MAPPING OF PROGRAM LEARNING OUTCOMES (PLOs)	VIII
CHAPTER # 01 INTRODUCTION	1
1.1. Problem Statement	2
1.2. Literature Review	3
1.3. Aims and Objective	4
CHAPTER # 02 METHODOLOGY	5
2.1. Hardware Detail	6
2.2. Software Detail	11
CHAPTER # 03 IMPLEMENTATION	14
3.1. Details of Hardware Implementation	15
3.2. Details of Software Implementation	46
CHAPTER # 04 PROJECT CHARACTERIZATION	53
4.1. Results	54
4.2. Analysis	57
4.3. Conclusion	58
4.4. Future Recommendations	59
REFERENCES	61
APPENDIX A: COMPLEX ENGINEERING PROBLEM	62
Appendix A1: Range of Resources	62
Appendix A2: Innovation	62
Appendix A3: Level of Interaction	63
Appendix A4: Consequences to Society and Environment	63
Appendix A5: Familiarity	63
APPENDIX B: TURNITIN REPORT	63

LIST OF FIGURES

Figure 2.1 Architecture of FPGA	6
Figure 2.2 Chipset hierarchy	7
Figure 2.3 IO CTRL TOP overview	7
Figure 2.4 AWS FPGA design	11
Figure 3.1 Openpiton system overview for quadcore	15
Figure 3.2 Block diagram of openpiton	16
Figure 3.3 Tile overview	17
Figure 3.4 Issue/ Read operands	19
Figure 3.5 Load store unit	20
Figure 3.6 Cache subsystem of ariane	22
Figure 3.7 Write through cache subsystem structure	23
Figure 3.8 L1 Data cache (tags, ways & sets)	25
Figure 3.9 L1 Implementation of tag array	26
Figure 3.10 Linear feedback shift register	27
Figure 3.11 Internal architecture of L1.5 adapter	28
Figure 3.12 L1.5 internal organization of address calculation	29
Figure 3.13 L1.5 internal organization of data cache	30
Figure 3.14 Internal organization of dcache and dtag	31
Figure 3.15 L1.5 internal organization of MSHR	32
Figure 3.16 Internal organization of RR LRU	33
Figure 3.17 Internal organization of way map table	33
Figure 3.18 L1.5 internal organization of MESI	34
Figure 3.19 L1.5 internal organization of tag organization	36
Figure 3.20 L2 pipelining flow	36
Figure 3.21 L2 pipeline 1 input buffer	37
Figure 3.22 L2 pipeline 2 input buffer	37

Figure 3.23 L2 directory array	38
Figure 3.24 L2 state array	38
Figure 3.25 L2 cache replacement/eviction algorithm	39
Figure 3.26 L2 tag array and tag hit	40
Figure 3.27 l2 data array	40
Figure 3.28 L2 MSHR	41
Figure 3.29 NOC router overview	42
Figure 3.30 NOC 1 wavefor	44
Figure 3.31 NOC flit overview	45
Figure 4.1 Thread running on each core	54
Figure 4.2 GTKWave form	54
Figure 4.3 Benchmarks regression testing results	55
Figure 4.4 Available global ID Of AFI	55
Figure 4.5 Output of runtime driver UART	56
Figure 4.6 Output of baremetal application	56
Figure 4.7 Booting linux output	57
Figure 4.8 Power analysis	58

LIST OF TABLES

Table 1 Utilization analysis of post implementation	59
Table 2 Range of resources	62

MAPPING OF PROGRAM LEARNING OUTCOMES (PLOs)

Chapters 1 through 4 and Appendix A & Appendix B	PLO-(x)-Communication PLO-(viii)-Ethics PLO-(iv)-Investigation
Chapter 01 – Introduction and Appendix A	PLO-(i)- Engineering Knowledge
Chapter 01 – Introduction	PLO-(ii)- Problem Analysis
Chapter 02 - Methodology	PLO-(iv)-Investigation
Chapter 03 - Implementation	PLO-(iii)-Design/Development of Solution
Chapter 04 – Project Characterization and Appendix A	PLO-(vi)-The Engineer and Society

CHAPTER # 01

INTRODUCTION

Introduction

This project involves generating a multicore, testing it on FPGA(field programmable gate array), and then booting an operating system on it. The transition between single core and the multicore processors has one significant difference which is a cache coherence operation that gets involved in multicore. This is the key element that must be taken care of for making any multicore processor as in the single core processor, all the resources especially memory is used by a single processor so it does needs any coherency been performed in that memory but that's not the case in multicore obviously due to the fact that the efficient multicore must be the one which should manage the usage of memory between all the processors effectively.

This project made the utilization of an opensource framework called openpiton for generating a cache coherence protocol, testing, and other processor designing activities. Since the important part of any SoC(system on chip) is the processor, So, it uses the ariane as its core because it is a well-known, tested, and efficient opensource RISC-V(Reduced instruction set architecture V) based core among others.

The project also aims to boot the SMP(symmetric multi-processing) linux operating system over that quad core which is the planned number of cores of the project.

1.1. Problem Statement

Most of the companies in the semiconductor industry like AMD, INTEL, APPLE, etc nowadays involve a large number of multicore chips based processors, but for anyone academically who wants to learn and experiment on multicore hands-on, does not have any useful opensource resources to get their

hands dirty. Since, the RISC-V is the new emerging processor architecture technology, it obviously doesn't have much work being done in the opensource multicore area.

OpenPiton addresses these issues and provides a full range of flexible and fully customized designing of a RISC-V based opensource multicore generation and can be taken till the tapeout as it is also been done by the makers.

This project is the demonstration of the development of a multicore processor through openPiton till the booting of the operating system which will also serve as a great learning resource for anyone wanting to understand the processes and the microarchitecture involved in a multicore processor and its development.

1.2. Literature Review

The RISC-V is a free and open source computer instruction set architecture (ISA) that is designed with well-organized modularity. It offers a high level of flexibility at a very low cost, allowing customers to create custom chips for specific applications. As Linux gained popularity in operating systems, RISC-V aimed to become a similar standard for processors and has been increasingly used in various commercial products[11].

OpenPiton+Ariane is an open source framework that is licensed in a permissive manner, and it enables the creation of scalable architecture research prototypes. It is the world's first open source, symmetric multiprocessing (SMP) system for RISC-V that can be used in both single-core and many-core configurations. This makes OpenPiton+Ariane an ideal platform for conducting hardware research related to RISC-V[14].

OpenPiton initially started as the world's first open source, general-purpose, multithreaded manycore processor and framework. It is built upon the reliable OpenSPARC T1 core but includes a custom scalable uncore (known as P-Mesh) to create a flexible and modern manycore design. OpenPiton provides a comprehensive verification infrastructure with over 80 thousand tests and is supported by mature software tools. It can run a full-stack multi-user Debian Linux operating system and is written in industry-standard Verilog hardware description language. Additionally, OpenPiton offers synthesis and back-end scripts for ASIC (Application Specific Integrated Circuit) and FPGA (Field-Programmable Gate Array) implementations, allowing other researchers to develop their own designs. Multiple implementations of OpenPiton have been created, including a 25-core implementation in IBM's 32 nm process and multiple Xilinx FPGA prototypes. Based on this mature foundation, OpenPiton+Ariane was developed. Ariane is a 64-bit RISC-V software processor that implements the RV64GC ISA (instruction set architecture)[13].

1.3. Aims and Objective

The project's aim is to design a multicore opensource SoC with full support from the software development kit. It could be used for an application class that can be used in a variety of areas including, Machine Learning IoT, Artificial Intelligence, and industrial application. However, making the design mature is time consuming, so this project initially will focus on designing a dual core processor and gradually adding other peripherals to make it a Multi-Core SoC for machine learning applications.

CHAPTER # 02

METHODOLOGY

2.1. Hardware Detail

The affordability and fast performance of FPGAs make them a popular option for verifying the hardware/software interface of a processor design. Openpiton provides support for a variety of FPGA boards.

2.1.1. Prototype Architecture of FPGA: Within openpiton, various scripts are available that facilitate the generation of FPGA bit files from Verilog source code. These scripts offer multiple configuration choices and enable the complete generation of bit files. The figure illustrates the top-level view of the prototype architecture specifically designed for FPGA, showcasing the default configuration of I/O devices.

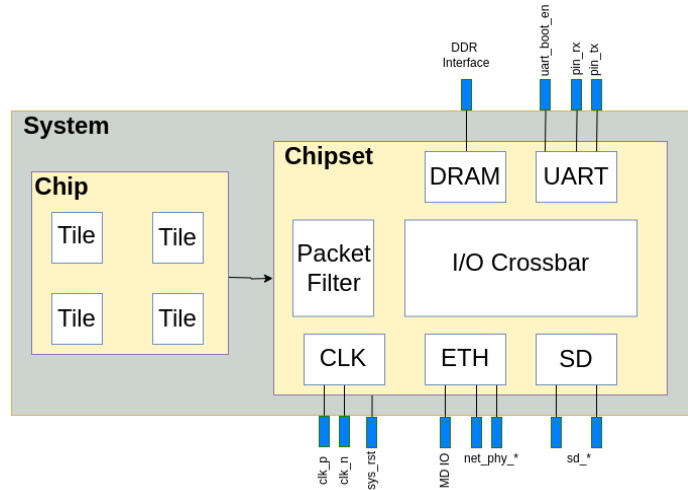


Figure 2.1 Architecture of FPGA

2.1.2. Chipset Internal Hierarchy: If we examine this module, namely the I/O crossbar and packet filter, in more detail, we will be able to see its internal hierarchy. This will provide us with a clearer understanding of how the I/O crossbar is being divided for different peripherals.

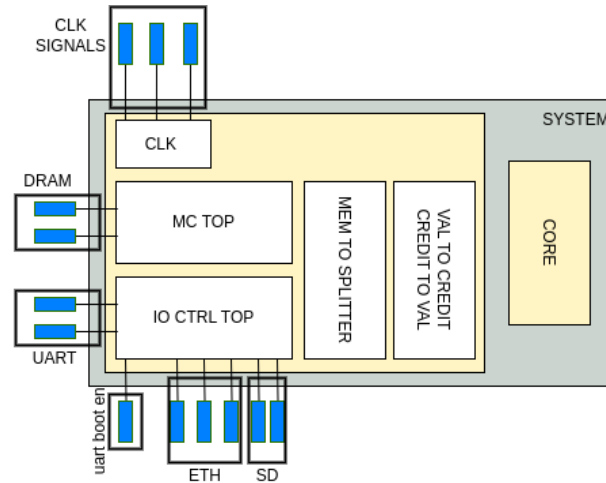


Figure 2.2 Chipset Hierarchy

The above image depicts the high-level architecture of the FPGA prototype. The top module of the FPGA, which is named "system.v," can be found in the directory path \$DC_ROOT/design/rtl.

2.1.3. IO_CTRL_TOP: When we dive deeper into the important module called "IO_ctrl_top," its significance lies in the fact that it facilitates NOC(Network on chip) to protocol conversion for various peripherals such as UART(Universal asynchronous receiver transmitter), ethernet, and others. The hierarchy of this module is crucial for understanding our FPGA design prototype. So, let's begin exploring this particular component.

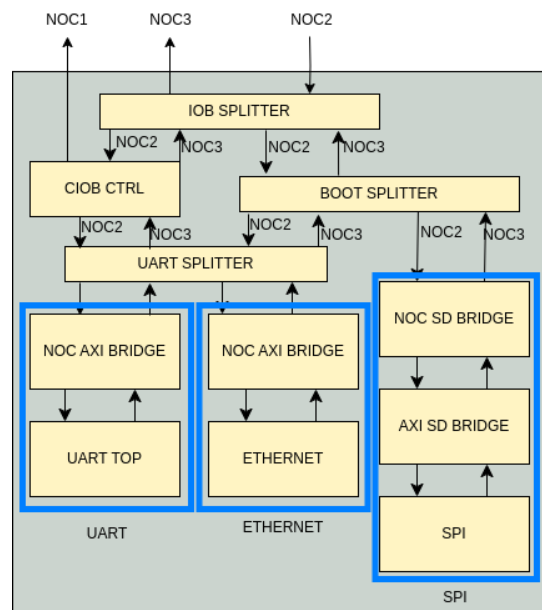


Figure 2.3 IO CTRL TOP Overview

As depicted in the above image, IO_CTRL_TOP utilizes the NOC1, NOC2, and NOC3 interfaces to establish communication with the chip. It uses an interface for SD card, UART(universal asynchronous receiver transmitter), and ethernet. Additionally, it incorporates the uart_tx/uart_rx interface for communication with a host PC via UART. The subblocks within this module are described in detail below.

- ciop_iob is a module that transmits interrupts to cores via NOC2. It is responsible for sending various types of interrupts, including a wakeup packet to a core as part of the reset sequence
- noc_axilite_bridge and uart_top are modules related to UART functionality. noc_axilite_bridge serves as a bridge between NOC and AXI(Advanced eXtensible Interface) Lite interfaces, facilitating communication between them.
- fake_boot_ctrl is a module that incorporates a Block RAM (BRAM) containing an assembly test, specifically designed for protosyn when the --bram-test option is enabled.

2.1.4. Hardware implementation for an FPGA: If we need to perform a hardware implementation for an FPGA, there are certain configurations and steps to consider. This includes the design bring-up and implementation process, as well as all the necessary tasks related to building the design for a specific FPGA[7].

The following steps outline the process, based on our understanding and experience of testing the design on the AWS (Amazon web services) cloud FPGA. The top-level hierarchy and the required steps for the aws cloud FPGA are considered, but similar commands can be used for other FPGA boards as well. Let's start with the process.

2.1.5. Configuration Usage:

```
protosyn -b <board_type> [-d <design>]  
[--bram-test <test_name>] [--from <FPGA flow step>]  
[--to <FPGA flow step>] [--no-ddr] [--uart-dmw ddr]  
[--eth] [--oled <string>]
```

-b, --board <board_type>

Name of a supported board. Available options are:

```
vc707, vcu118, xupp3r, genesys2, nexysVideo, fl -d, --design <design>
```

Name of design module to synthesize. The default is 'system', which synthesizes a full system with a chip and chipset. See \$DV_ROOT/tools/src/proto/block.list for supported design modules

-c, --core <core_config>

Name of a supported core configuration:

- *sparc* (opensparc T1 core, default)
- *pico* (32bit RISCv core)

- *pico_het* (*heterogeneous pico+sparc arrangement*)
- *ariane* (*64bit RISCv core*)

--network_config <network_config>

Name of the network type to be used:

- *2dmesh_config* (*default*)
- *xbar_config*

--x_tiles <number>

--y_tiles <number>

Number of tiles to instantiate in each direction. Default: 1.

--num_tiles <number>

Total number of tiles. Default: 1.

To synthesize openpiton for AWS F1 instances, there are several steps to follow. The overall flow is similar to synthesizing an image for any other supported FPGA.

For generating AWS-FPGA EC2 bitstream, steps can be found here [10].

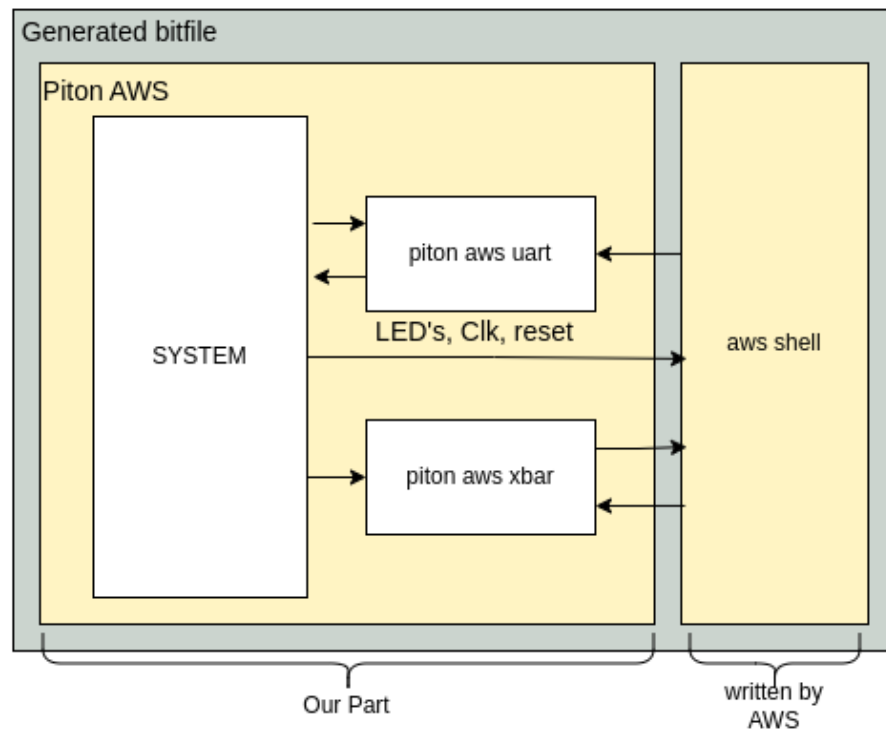


Figure 2.4 AWS FPGA Design

The upper design refers to the design that is generated as a bit file specifically for the AWS FPGA F1 board, which is then loaded onto the F1 board for further use.

2.2. Software Detail

2.2.1 Openpiton:

Users have access to a comprehensive method for validating and verifying the correctness and performance of the openpiton due to the availability of a comprehensive test suite that includes unit tests, single core tests, and multicore tests.

Openpiton's capacity for scalability opens up a large amount of applications in a variety of fields. It has the potential to change high-performance computing, data centers, scientific research, artificial intelligence, and a lot of other things that need a lot of computational power.

The 2D cross section engineering utilized by openpiton upgrades adaptability, as it works with effective correspondence and information moving between centers. In addition, openpiton is an excellent educational resource for teaching and learning about the design of advanced processors and multicore architectures[3].

The openpiton project has the potential to shape the future of manycore processors and enables groundbreaking discoveries and innovations in the field of high performance computing as it continues to develop and gain traction in the research and industry communities.

2.2.2. Xilinx Vivado: Xilinx's Vivado Design Suite is a comprehensive software suite for synthesizing and analyzing hardware description language (HDL) designs. It provides a wide range of additional features specifically designed for SoC development and high-level synthesis, making it an advanced replacement for the Xilinx ISE(Integrated Software Environment) software. Vivado addresses a critical jump forward in the plan stream contrasted with its ancestor, ISE. The entire design process is rewritten and rethought from the ground up in order to introduce a more modern and effective HDL design method.

Vivado's built-in logic simulator is a standout feature because it lets designers simulate and validate HDL designs in the software environment. Designs can be thoroughly tested and verified using this integrated simulation capability, ensuring their accuracy and functionality prior to implementation. High-level synthesis (HLS), a powerful toolchain for converting C code into programmable logic, is also introduced in Vivado. With the assistance of this

creative development, originators can work at a more prominent level of reflection, which helps efficiency and empowers speedier plan cycles.

2.2.3. Verilator: Verilator is a tool that can be used to verify and simulate Verilog or SystemVerilog code in a variety of ways. By "verilating" the predetermined code, verilator performs different undertakings to guarantee the code's uprightness and usefulness.

Verilator also lets you add coverage-analysis points and assertion checks to the code. These enable more in-depth design testing and analysis, assisting in the identification of flaws or areas in need of improvement.

The simulation can be carried out using the resulting Verilated model, allowing designers to evaluate their designs' behavior and performance. This recreation interaction gives significant experiences into the plan's usefulness, considering iterative enhancements and refinements.

CHAPTER # 03

IMPLEMENTATION

3.1. Details of Hardware Implementation

The plan contains an overview of the system, a brief description of the major tasks involved in the implementation, and the overall resources needed to support the implementation effort (such as hardware, software, facilities, materials, and personnel), and any site-specific implementation requirements.

3.1.1. Overview: In the context of openpiton, the architecture is designed to be scalable both intra-chip and inter-chip. This means that openpiton is designed to be highly configurable.

3.1.2. Intra-Chip: Intra-chip scalability refers to the ability of a processor architecture to scale within a single chip. This includes the ability to add more cores, cache, memory controllers, and other components. In openpiton, intra-chip connectivity is achieved through a 2D mesh topology [5].

3.1.3. Inter-Chip: Inter-Chip refers to the ability of a processor architecture to scale across multiple chips.

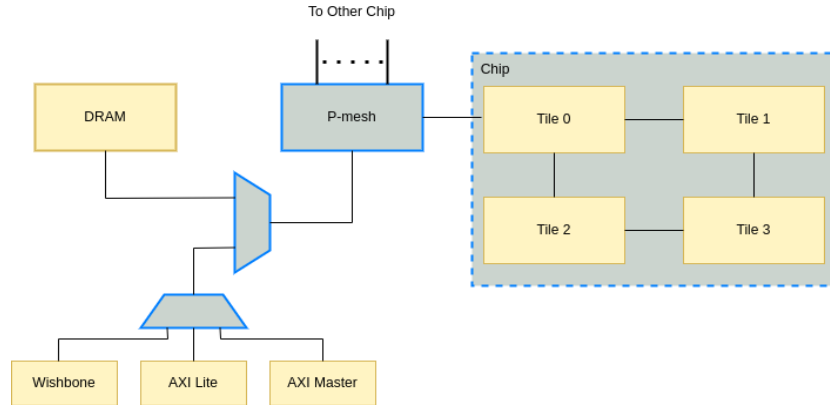


Figure 3.1 Openpiton System Overview for quadcore

In the context of openpiton, the chipset refers to the off-chip logic(outside chip) that is connected to the tiled manycore array via the chip bridge to communicate with other components in the system, such as memory, storage, and network interfaces[5].

3.1.4. Block Diagram: In the block diagram, it is shown that the top left module, representing a tile, is opened and defined. Similarly, an identical clone of this module is present on the top right side, which contains the modules for ariane and PMesh, which are exactly the same as the first module. This pattern continues downwards as well. Similarly, on the left side of this module, four tiles are communicating with each other in a certain way. Then these connections extend outside to connect with the chipset, and after that, the chipset is connected to all the peripherals [5].

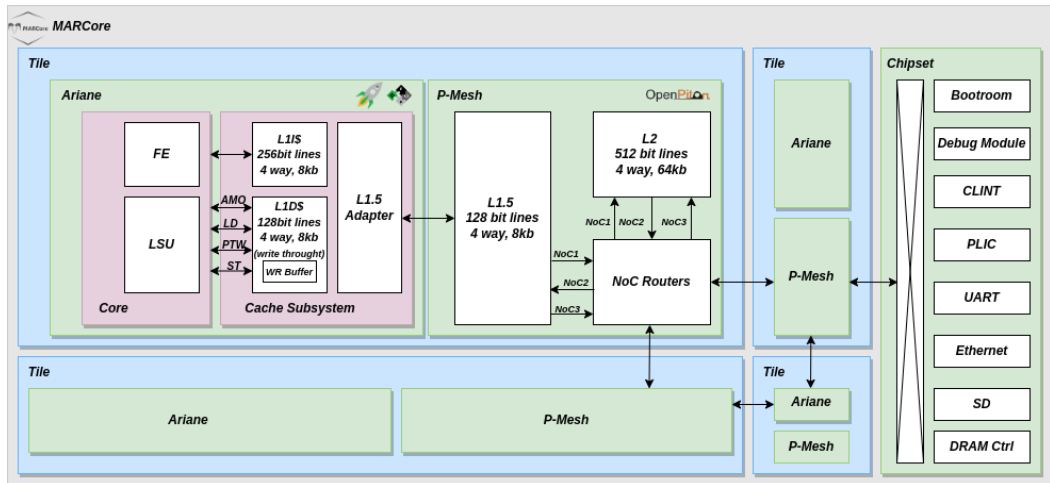


Figure 3.2 Block Diagram of openpiton

3.1.5. Peripheral Base Addresses: In openpiton, various peripherals are connected, such as ROM, Debug module, CLINT (Core Local Interrupt), PLIC (Platform Level Interrupt Controller), UART, ethernet, SD card, and DRAM. Some peripheral addresses are specified here. However, the regions and base addresses of other peripherals are defined in a directory. `"/openpiton/piton/design/xilinx/f1/devices_ariane.xml"` and `"devices.xml"`[7].

3.1.6. Tile Overview: A tile is made up of several parts, including a modified ariane core, an L1.5 cache, an L2 cache, an L1.5 arbiter, and three NoC routers. The L2 and L1.5 caches are directly connected to all three NoC routers, and any messages going in or out of the tile use these connections.

To connect the core and L1.5 cache in ariane, they use a transaction response interface (TRI). The L1.5 cache has a specific role of converting information between the PCX decoder and the NoC router protocol..

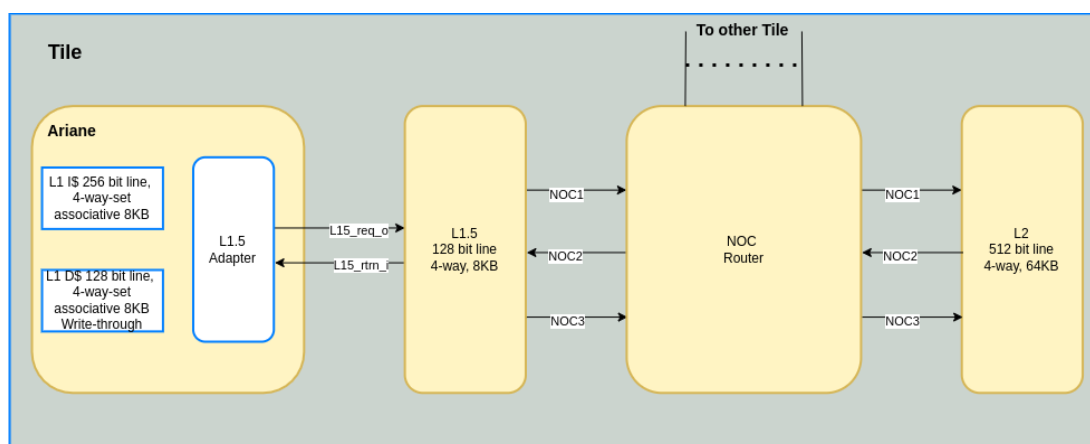


Figure 3.3 Tile Overview

Now, the description of all the modules within this tile is provided below. For example, there is a description of the core, followed by the cache subsystem of the core, the native interface of this core, and L1.5 and L2, which is the distributed cache of the openpiton architecture [5]. The modules within L2 are also further described, such as the replacement policy being used. If it is LRU (Least Recently Used), there is an explanation of how LRU works. If it is LFSR (Linear Feedback Shift Register), there is an explanation of what LFSR is and how its algorithm operates. All of these explanations are provided in detail below.

3.1.7. Ariane: It is the 6-stage pipelined, single issue CPU which implements the 64-bit RISC-V instructions. It fully implements I, M, and C extension. It implements three privilege levels M, S, and U to fully support a Linux operating system.

The **PC Gen stage** is responsible for generating the next program counter (PC) in a processor. It handles logically addressed program counters. This stage contains speculation on the branch target address and whether the branch is taken or not. PCGen communicates with the instruction fetch which is the next stage using a handshake signal. it signals this readiness with an asserted ready signal, while PCGen indicates a valid request by asserting the fetch_valid signal.[12].

Instruction Fetch stage (IF) gets information from the PC Gen stage. It initiates the address translation on the requested Program Counter (PC) by interacting with the Memory Management Unit (MMU) and takes charge of controlling the instruction memory interface. The sensitive part of the instruction fetch is timing constraints. This fact prevents us from implementing some more elaborate handshake protocol. Hence, the IF stage informs the I\$ interface that it wants to make a fetch request to memory.

The fetch FIFO has all the requested fetches from the instruction memory. The FIFO currently has one write port and two read ports. Moreover, the fetch FIFO serves a vital role in fully decoupling the front-end and back-end of the processor. On a flush request the whole fetch FIFO is reset ensuring that any speculative or incorrect instructions are cleared.

Instruction decode stage is the first pipeline stage of the processor's back-end. Its primary objective is to extract instructions from the data stream it recieved from IF stage, decode them and forward them to the issue stage.

However, the introduction of compressed instructions, which are generally of variable length, adds complexity to the ID stage. It has to search the incoming data stream for potential instructions, re-align them and in the case of compressed instructions decompress them. To decompress all the compressed instructions a small combinatorial circuit which takes a 16-bit compressed instruction and expands it to its 32-bit equivalent. All compressed instructions have a 32-bit equivalent. This process allows the processor to effectively work with compressed and uncompressed instructions, ensuring smooth execution regardless of the instruction format.

The **Issue stage**'s purpose is to receive the decoded instructions and issue them to the respective functional units. Additionally, the issue stage keeps a record of all issued instructions, monitors the status of the functional unit and receives the write-back data from the execute stage. And it also contains the CPU's register file. The Issue stage employs a data structure called a scoreboard. The scoreboard allows the stage to precisely track the instructions that have been issued, the specific functional unit each instruction is assigned to, and the register to which they will write back the results. This real-time information enables effective instruction scheduling and ensures that dependencies are properly handled, avoiding data hazards and potential conflicts during execution. As previously mentioned you can roughly divide the execution into four parts:

- Issue
- Read operands
- Execute
- Write-back

The **Execute stage** is a logical stage that encapsulates all the functional units (FUs). Currently, these FUs operate independently without any inter-unit dependencies. Each functional unit possesses a valid signal, indicating the availability of valid output data, along with a ready signal to communicate its ability to accept new requests to the issue logic.

The arithmetic logic unit (ALU) is a compact hardware component designed to carry out various operations, such as 32 and 64-bit subtraction, addition, shifts, and comparisons. Remarkably, the ALU executes its tasks in a single cycle, making it stateless without any stateful elements. Consequently, its ready signal is consistently asserted, and it directly transfers the transaction ID from its input to its output. Alongside the two operands, it also receives an operator that instructs it on the specific operation to perform.

The primary function of the branch unit is to handle various control flow modifications, encompassing both conditional and unconditional jumps. To achieve this, it incorporates an adder to compute the target address and comparison logic to determine whether the branch should be executed or not. Additionally, the branch unit is responsible for identifying mis-predicted branches. When a mis-prediction occurs, it reports the necessary corrective measures to the PCGen stage, which may involve updating the Branch History Table (BHT) and adjusting the Program Counter (PC) as required.

The load store unit is similar to other functional units. However, it has the additional responsibility of overseeing the interface to the data cache. Specifically, it contains the DTLB (Data Translation Lookaside Buffer), the hardware page table walker (PTW), and the memory management unit (MMU). It also arbitrates the access to data memory between loads, stores and the PTW -

giving precedence to PTW lookups. This prioritization is crucial to promptly resolve TLB (Translation Lookaside Buffer) misses and optimize memory access.

A high level block diagram of the LSU is following:

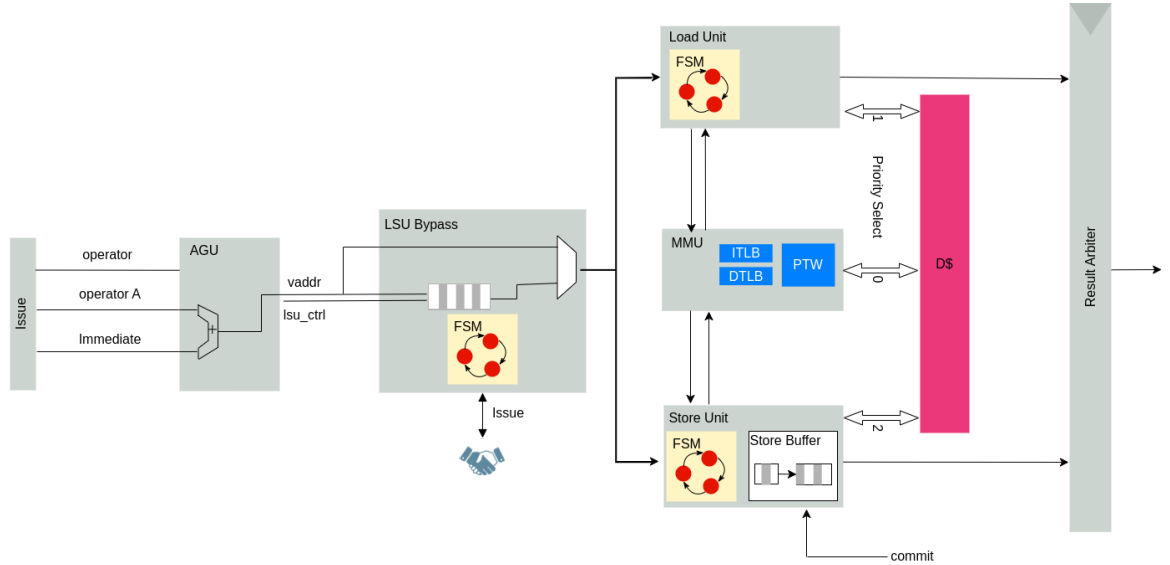


Figure 3.5 Load Store Unit

To ease the design of the LSU it is split in 6 major parts of which are:

- LSU Bypass- manages the LSU status information which it presents to the issue stage.
- D\$ Arbiter- managing access to the data memory
- Load Unit- perform address translation, takes care of all loads
- Store Unit- manages all stores, calculates the target address and setting the appropriate byte enable bits
- MMU (including TLBs and PTW)- prioritizes instruction address translations to data address translations.
- Non-blocking data cache- allows multiple memory access requests to be handled independently and in parallel.

The **Commit stage** is the last stage in the processor's pipeline. Its purpose is to receive incoming instructions and update the architectural state. This includes writing CSR registers, committing stores, and writing back data to the

register file. The golden rule is that no other pipeline stage is allowed to update the architectural state under any circumstances. Additionally, if the Commit stage maintains any internal state, it must be designed in a way that allows it to be reset when necessary to ensure consistency and reliability.

3.1.8. Cache Subsystem: In Ariane, there are two different types of modules that are used as a Cache Subsystem: write-through cache subsystem and write-back cache subsystem. The default module used for the Cache Subsystem is the write-through cache subsystem, which is named "wt_cache_subsystem".

However, if developers need to use the write-back cache subsystem, then a different module called "std_cache_subsystem" is used. This module description is the standard Ariane cache subsystem and has an instruction cache and a write-back data cache [1] . These top modules description are as follows.

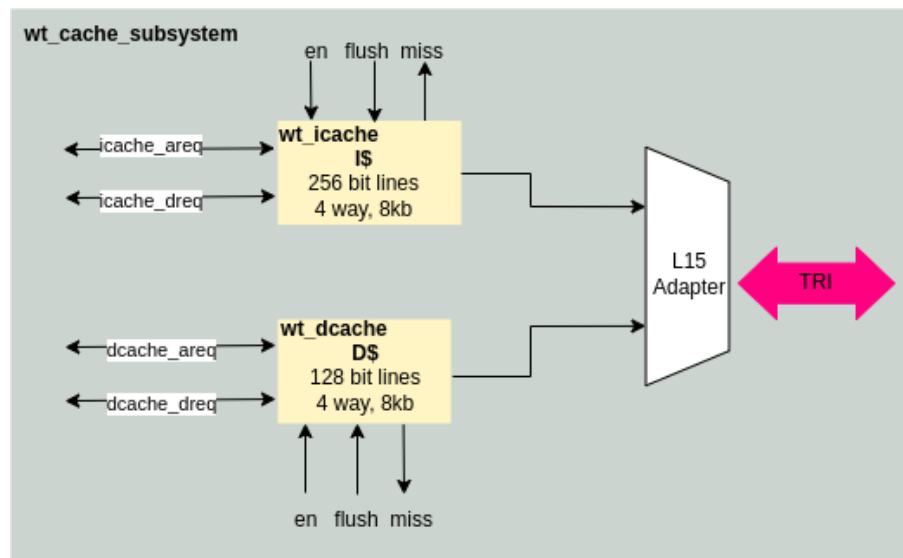


Figure 3.6 Cache Subsystem of ariane

a) **L1 Instruction Cache:** The instruction cache in ariane is responsible for storing and providing quick access to frequently used instructions fetched from memory.

Calculations of I\$

- **Size Of Instruction Cache:**

$$\text{Cache Size} = \text{Total no. of sets} * \text{ways in each set} * \text{No. of bits in one way}$$

$$\text{Cache Size} = 64 * 4 * 256 = 8\text{KB}$$

b) **L1 Data Cache:** Ariane's data cache is essential for storing frequently requested data in memory.

Write Through Cache Subsystem: The write-through cache subsystem of ariane has a top module called "wt_cache_subsystem." This top module contains three main modules: "wt_icache" for L1 Instruction Cache, and "wt_dcache" for L1 write-through data cache and arbiter.

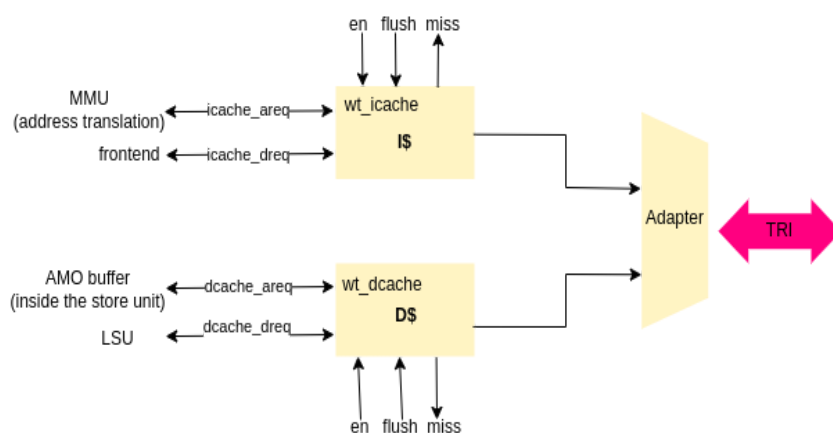


Figure 3.7 Write Through Cache Subsystem Structure

Internal Organization of Data Memory

Calculations of D\$

- **Size Of Data Cache:**

$$\text{Cache Size} = \text{Total no. of sets} * \text{ways in each set} * \text{No. of bits in one way}$$

$$\text{Cache Size} = 128 * 4 * 128 = 8\text{KB}$$

a) Case Of Read Hit: If a read request is hit, the cache controller may instantly send a new request if one is available.

b) Case Of Write Hit: The cache controller enters the STORE_REQ state, waits for the cache content update to be finished, and then returns to the IDLE state.

c) Case Of Miss: When the controller receives a request for data, it sends a request to the miss handler which interfaces with the outside world. The controller goes through different states like WAIT_REFILL_GNT to wait for data fetch and IDLE, WAIT_CRITICAL_WORD, or WAIT_REFILL_VALID based on the type of request.

d) DCache Address Decomposition: Considering the previously supplied structure, it is now possible to specify how to decompose a particular address so one can find which location within the cache it represents. For the ariane L1 facts cache, the physical addresses are 64-bit long and are composed of the subsequent fields:

- **Offset bits (0-3):** The offset specifies the location of a byte inside a cache line, helping pick the precise records bank (data bank).
These 4 bits allow for addressing individual bytes within a cache line.
- **Index bits (4-11):** The index bits distinguish among the to be had sets in the cache. In this example, there are 128 sets within the cache.
- **Tag bits (12-63):** The tag field consists of 52 bits and is used to represent the requested memory location. It uniquely identifies the memory block that the address corresponds to.

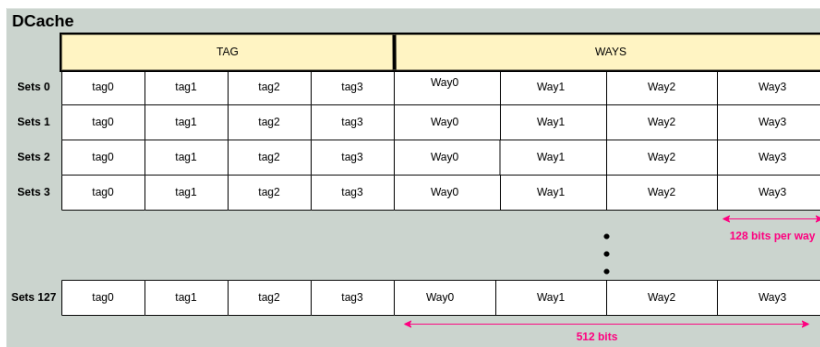


Figure 3.8 L1 Data Cache (Tags, Ways & Sets)

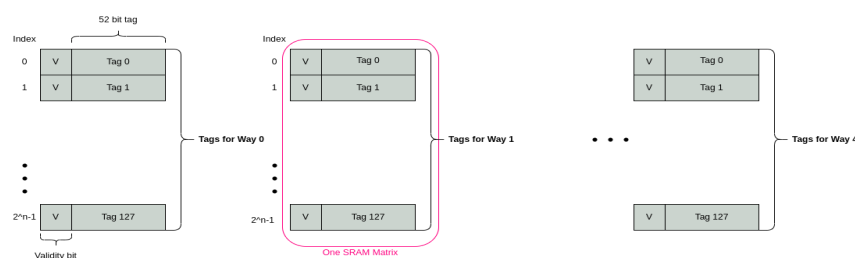


Figure 3.9 L1 Implementation of Tag Array

To determine which bank in the data array to target, the MSB of the offset is used:

The bank 0 will be chosen when the offset is between 0 and 3, as the MSB is equal to 0.

The MSB is equal to one for the remaining values (8 to 15), so bank 1 will be chosen.

The tag is 0x0000008000b in format. The SRAM matrix entry number for the corresponding tag will reveal the direction that the address being considered is targeting among the previously chosen tags. The tag refers to way number three because the entry from the four SRAM matrix is also equal to 0x0000008000b. The requested 64-bit memory block is located at address 0x0000008000b010 as a result.

Replacement Policy of L1 Cache: The Linear Feedback Shift Register replacement policy is used in Ariane's L1 Cache. The Linear-Feedback Shift Register (LFSR) is a shift register in which the input bit is produced by performing an XOR operation on a subset of the register's bits. When a cache line needs to be removed, LFSR is a popular method of generating pseudorandom numbers that produce a pseudorandom output. When a cache miss occurs while the cache is full, this eviction policy is applied.

The eviction policy alters, though, if empty cache lines with their valid bit set to zero are available. The line with the lowest index, which has its validity bit set to 0, is chosen as the first cache line with a non-valid bit in this situation.

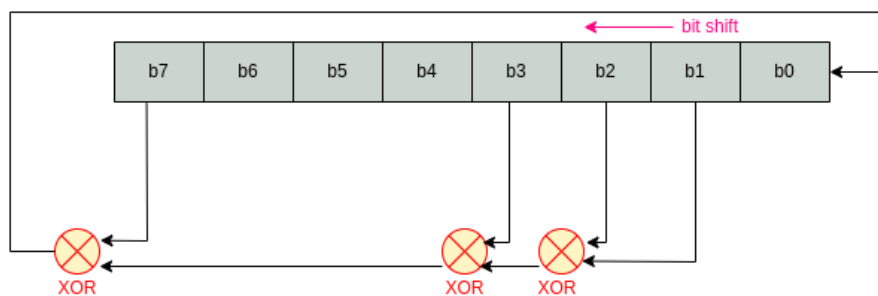


Figure 3.10 Linear Feedback Shift Register

Transaction Response Interface: The P-Mesh cache subsystem of openpiton connects to the L1.5 cache of the core L1 caches via the TRI. It is made to make data transfer and communication between these components easier. For cores following a write-through cache protocol, TRI is a generic and straightforward interface. It supports functions like atomics and invalidation messages.

L1.5 ADAPTER: The write-through L15 adapter of ariane has been named as "wt_l15_adapter." L1.5 Adapter is designed to connect the L1D\$ (data cache) and L1I\$ (instruction cache) to the native interface of the openpiton L1.5 cache. It essentially handles the communication between the ariane core and the L1.5 cache. It receives cache requests from the instruction and data caches, selects the appropriate request to be sent to the L1.5 cache based on priority and request type, and forwards the requests along with necessary data and control [5].

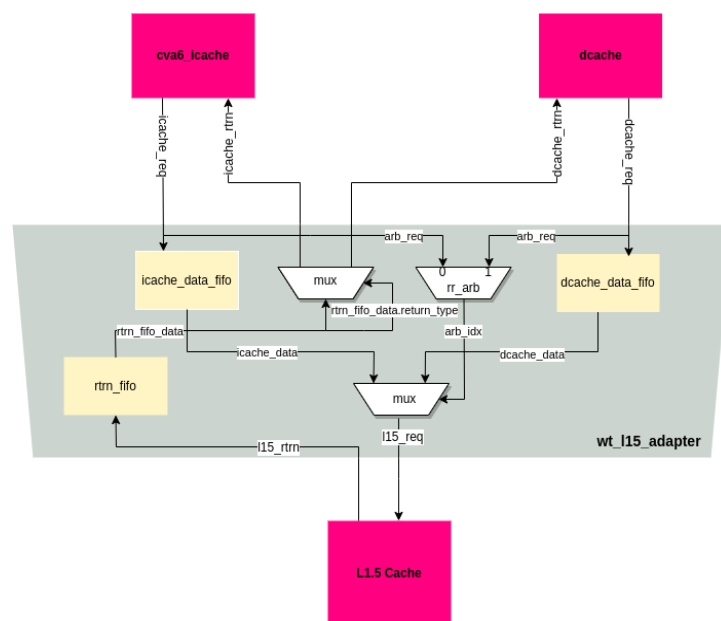


Figure 3.11 Internal Architecture of L1.5 Adapter

The figure 3.11 shows the internal structure of the write through l15 adapter.

A detailed explanation of each module of l15 adapter is given below:

3.1.9. L1.5: Now we will discuss L1.5 Cache of openpiton. It's Configuration, Structure, hierarchy and internal design organization. Therefore, within openpiton, the instruction cache lines are not stored in L1.5 cache. L1.5 cache does not monitor the L1-I cache for instructions. Instead, L1.5 cache is inclusive in L1-D cache for data. When it comes to cache size and configuration, L1.5 cache is completely independent of both L1-D and L2 caches. By default, the L1.5 is 8KB in size and 4-way associative. Communication between these two caches is done with the TRI(Transition Response Interface) bus interface.

Address calculation Of Tag Ram: L1.5 is a 4-way set associative cache with 128 bits per line. The total sets are 128 and there are 4 ways in each set. So, for selecting the address of a set in D-cache there are 7 bits required and 2 bits are for selecting from the 4- ways (Way0-3) in a set. So there are a total of 9 bits required as an index addressing of cache memory. Each way contains 4 words and each word is 32 bit of size So the actual 7 bits index addressing of cache is left shifted by 2 for way offset.

$$\text{Cache Size} = \text{number of sets} * \text{associativity} * \text{block size} = 128 * 4 * 128 = 64 \text{ Kb} = 8 \text{ KB}$$

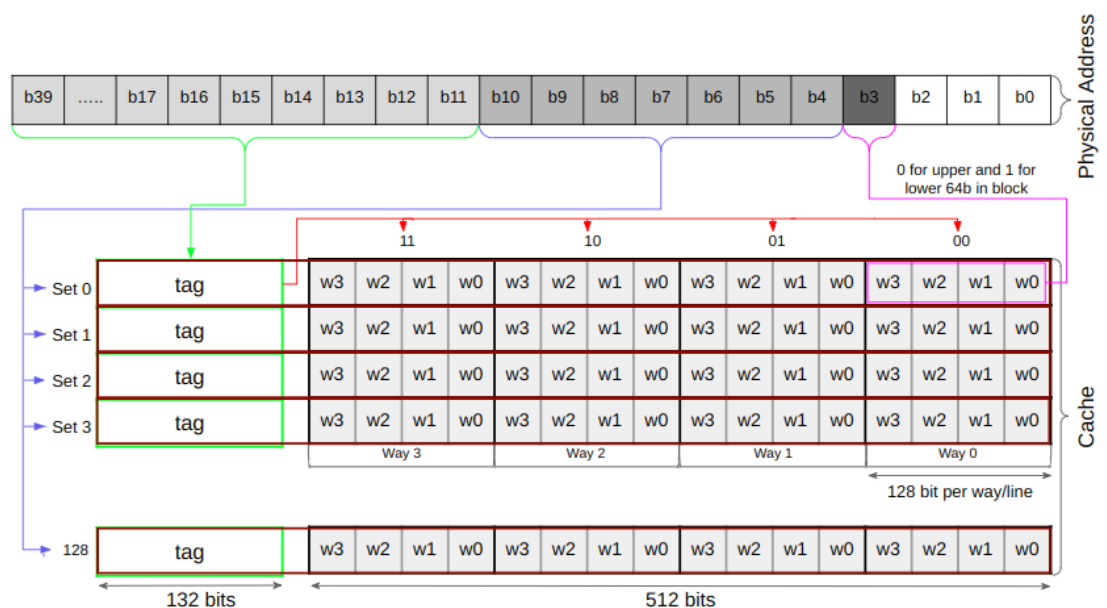


Figure 3.12 L1.5 Internal Organization Of Address Calculation

Internal Organization of Data Cache: Dcache is used for 4-way set-associative cache, its dimensions are 512 (depth) x 128 (width). There are a total of 128 sets in it and there are 4 ways and each way is 128 bits. So 128×4 is the set size. So a bit per line is equal to 128 bits. So for Dcache size, we have block size = 128 bits, number of sets = 128, and associative size = 4. In each block, there are 4 words and each word size is equal to 32 bits.

$$\text{Cache size} = \text{number of sets} * \text{block size} * \text{associativity} = 128 \times 128 \times 4 = 64 \text{ kilobits} = 8KB$$

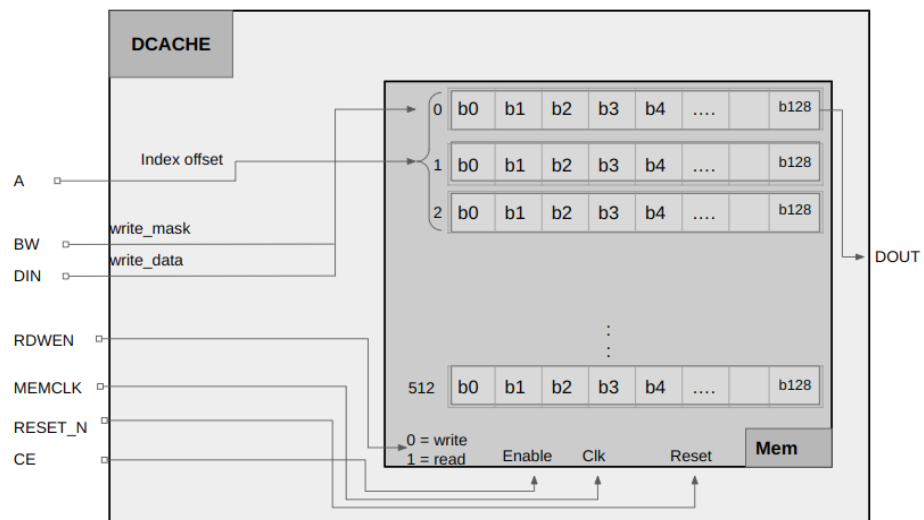


Figure 3.13 L1.5 Internal Organization of Data Cache

Internal Organization of Tag Memory: Dtag is used for 4-way set-associative cache, its dimensions are 128 (depth) x 132 (width). So, each tag-check way equals = 33 bits of which 29 bits are used for the tag. Dtag is a write mask in which 7 bits are used for the address index. 'A' is the interface of it. Each 29 MSB's physical address is the tag or the data of the Dtag. There is one bit for chip enable. RDWIN for read and write operation 1 for read and 0 for write. There is a BW port which is for writing masks. The input data bit size is 132 bits.

So, 29 bits of the physical address are written in a 4-way with a write mask. The output port size of Dtag is 132 bits.

Internal Organization of Dcache and Dtag: This diagram is for comparison purposes and clearly illustrates how the organization of memory, whether it is tag memory or data memory, works. For example, if I fetch data from the tag memory using a specific address, I will obtain four different tags corresponding to different locations. This makes it easier to determine which way will result in a hit or a miss.

This is the reason why tag memory is accessed in the first stage, and data memory is accessed in a later stage. As a result, the least significant bits, which are determined by the initial stage, indicate which way to access. Then, based on this information, there is data for one way per address, which can be successfully accessed.

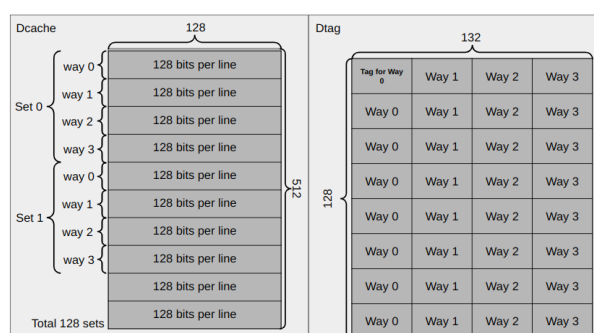


Figure 3.14 Internal Organization of DCache and Dtag

L1.5 MSHR(Miss Handling Status Register): When a cache miss occurs, meaning the requested data is not present in the cache, the MSHR is responsible for initiating the necessary actions to retrieve the data from the next level of the memory hierarchy (e.g., higher-level cache or main memory). It keeps a record of the missed address, the type of operation (read or write), and any additional metadata required for handling the miss.

Furthermore, in a multithreading environment, where multiple threads or tasks are executed concurrently within a single core, the MSHR is responsible for ensuring proper synchronization and coherence. It ensures that each thread's cache misses are tracked and handled independently, preventing interference or incorrect data access between threads.

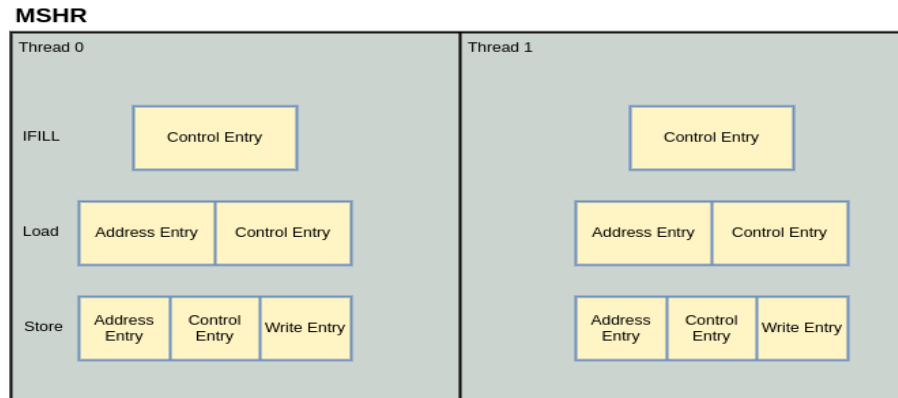


Figure 3.15 L1.5 Internal Organization of MSHR

Replacement Policy (LRU, RR “Round Robin”)

The 4 bits for LRU are used to track the access history of cache lines. Each cache line has its corresponding 4-bit LRU counter. When a cache line is accessed, its LRU counter is updated to reflect the most recent access. The LRU counter bits are typically assigned binary values, such as 00, 01, 10, and 11, representing different levels of recency. The cache line with the highest binary value indicates the least recently used line, and therefore, it is a potential candidate for eviction.

The 2 bits for Round Robin serve as a way to cycle through cache lines and ensure fairness in eviction. Each cache line has its 2-bit Round Robin counter. When a cache line is accessed, its Round Robin counter is updated to the next value in the round-robin sequence. The round-robin sequence typically includes binary values 00, 01, 10, and 11, which are cyclically assigned to cache lines in a predetermined order.

When a cache eviction is required, both the LRU and Round Robin counters are examined. The cache line with the highest LRU counter value among those with the same Round Robin counter value is selected for eviction. This combination ensures that cache lines that have not been accessed recently are more likely to be evicted, while also taking into account the fairness provided by the Round Robin counter.

Round Robbin LRU

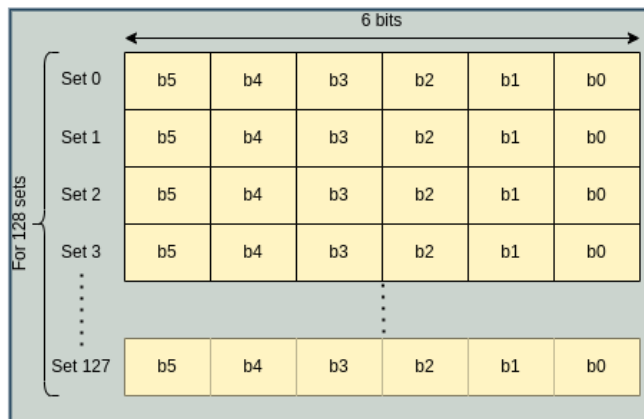


Figure 3.16 Internal Organization of RR LRU

Way Map table: The "way map table" is another module specific to L1.5 cache, which is used to store a valid bit for each set. This table ensures that for each address, there are 4 bits in the map table corresponding to the 4 ways in the cache. These bits indicate the validity of each way in the cache set.

Way Map Table

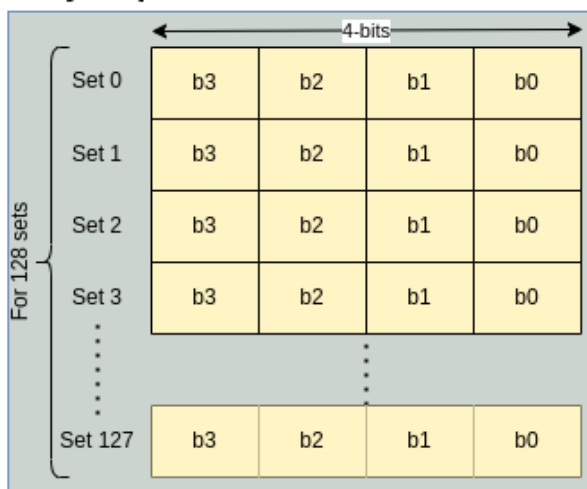


Figure 3.17 Internal Organization of Way Map Table

MESI Protocol: In this block diagram, we can observe that there is a separate module for storing MESI (Modified, Exclusive, Shared, Invalid) data, which is shown in the diagram as the "MESI Module." In this module, the memory organization is depicted, where for each address, there are MESI bits saved for 4 different ways: way 0, way 1, way 2, and way 3. This allows the system to track the MESI state for each way in the cache.

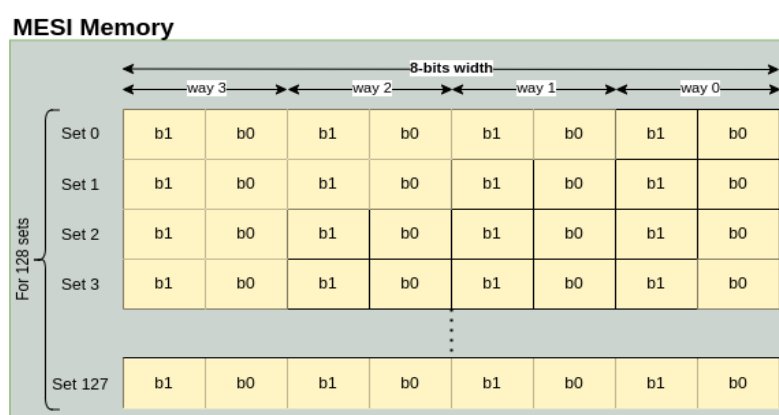


Figure 3.18 L1.5 Internal Organization of MESI

Invalid to Exclusive: In this test scenario, a load operation is performed from core 1 to, for example, check if the data in its private memory is invalid. If the data is invalid in the private memory, core 1 will request the data from shared memory. If the data is also invalid in shared memory, core 1 will then request the data from the main memory. When the data returns from the main memory to the shared memory, the MESI bits will transition from invalid to exclusive. Similarly, the data in the private cache will transition from invalid to exclusive.

Invalid to modify: This is another test scenario for coherence transactions. In this scenario, if core 2 wants to store some data, which we refer to as the "modified" state in MESI, it cannot directly modify it. First, it checks core 1 to see if there is already exclusive, shared, or modified data present. If there is, it invalidates that data first for the respective core. According to the diagram, the invalidation will occur in core 1 first. Once the invalidation acknowledgment is

received, the data in core 2 will transition from invalid to modified. The observation here is that it takes 4 hops to perform the modification.

L1.5 pipeline: There are two separate memories for tag and data called Dtag and Dcache. These two modules are tightly integrated with another module named pipeline. The read/write operations between the l1.5 and core are done in the 3 stages called S1, S2, and S3. Therefore, within openpiton, the instruction cache lines are not stored in the L1.5 cache. L1.5 cache does not monitor the L1-I cache for instructions.

Instead, the L1.5 cache is inclusive of the L1-D cache for data. When it comes to cache size and configuration, the L1.5 cache is completely independent of both L1-D and L2 caches. The core sends requests to S1, which then decodes them. S2 performs a tag check and sends read and write commands to the data SRAM. Finally, S3 directs the data and sends requests to the appropriate output buffer.

Tag hit/miss organization: This figure illustrates the logic path for tag hit and miss in a private cache, showing how a hit or miss occurs. The physical address's tag bits are compared to the full width of the tag memory, ensuring an address match with the tag. The resulting output is then logically ANDed with the valid bit, which is stored in the way map table. If the data is valid and the tag matches, we consider it a hit. After a hit, the next step is to select the appropriate way, which involves decoding and selecting the corresponding way for further use.

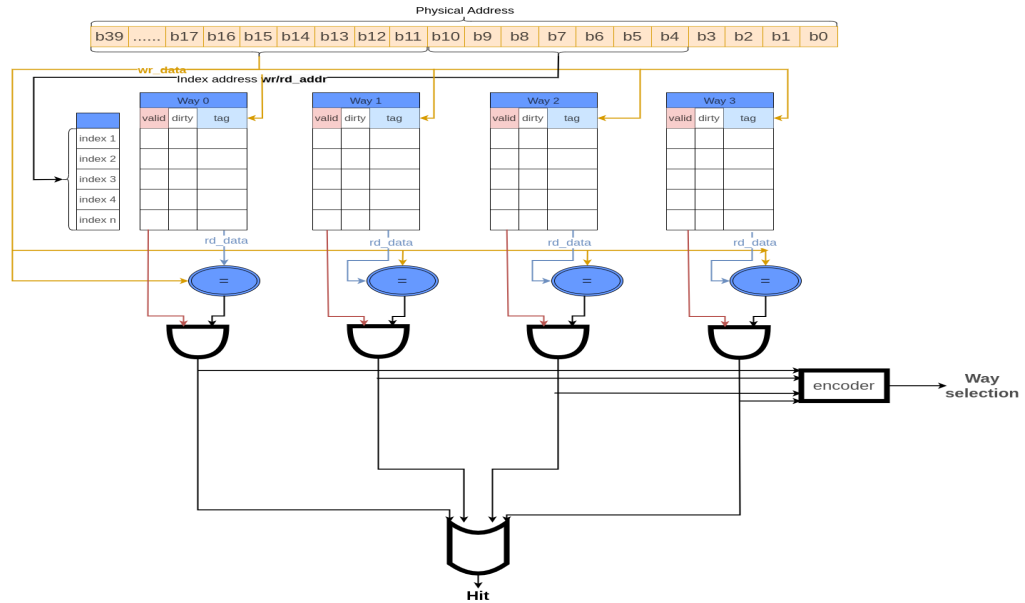


Figure 3.19 L1.5 Internal Organization of Tag Organization

3.1.10. L2: The L2 cache is a distributed write-back cache shared by all cores. The pipeline flow is explained below:

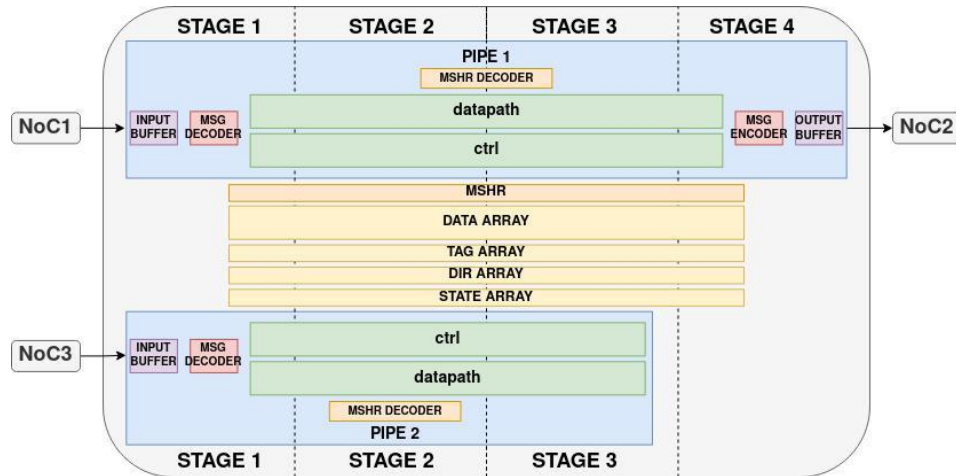


Figure 3.20 L2 Pipelining Flow

- Each of the 2 pipelines (pipeline1 for noc1, pipeline2 for noc3) has input and output buffers at pipeline entry and exit points.
- The first pipeline contains 4 stages and the second pipeline contains only 3 stages (Because it does not need to send output messages).
- Since most sub-modules only have a read/write port, they cannot be accessed by both pipelines at the same time. Therefore, if one stage in one pipeline is active, the corresponding stage in the other pipeline must be stalled.

(this is done by adding busy state signals coming from pipeline 2, and if i.e state 2 is active in pipeline 2 then 2nd state in pipeline 1 gets installed).

INPUT buffers

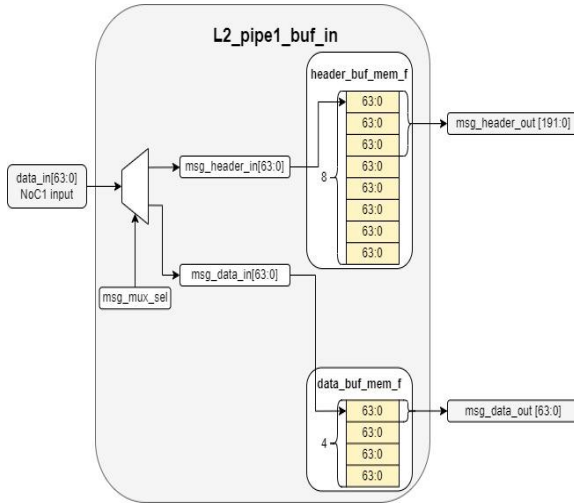


Figure 3.21 L2 pipeline 1 input buffer

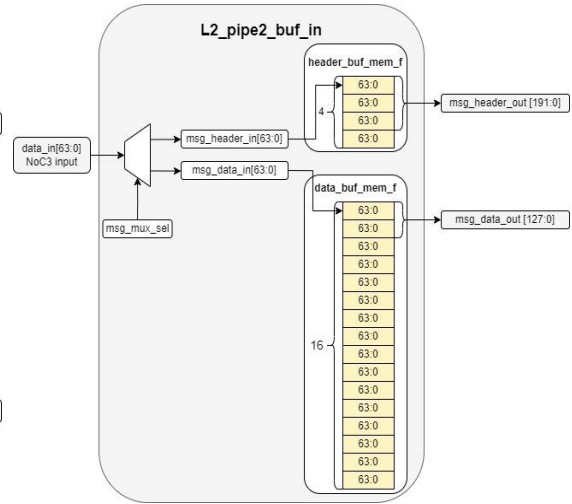


Figure 3.22 L2 pipeline 2 input buffer

Input buffers separate the input as data or messages. There are 8 flits as the header buffer size and 4 flits as the data buffer size because most of the input messages from NoC1 do not contain data. While for NoC3 input buffer, we choose 4 flits as the header buffer size and 16 flits as the data buffer size because NoC3 input messages include the memory response which contains 8 flits of data.

MESI: Open piton uses directory based MESI protocol. Since it is a big multi-core framework, it cannot use snooping-based cache coherence in which every share must snoop on the shared bus for the shared data which makes big multicores hard to do so.

It stores each tag-related state information (Modified, exclusive, shared, Invalid) in state array memory.

3.1.11. Types of memories(arrays) in the L2 cache:

Directory array: This is used to store the sharer information for each cache line. The reason behind using directory-based MESI in open-piton is because It has a 64-bit data line so it can store up to 64 bits of shared information respective to each tag. It takes 10 bits of address information which contain 8 bits of tag and 2 bits for way selections. (10x64b)

So, each dir address contains up to 64 sharer's information for each cache line in the data array.

The sharer information(input data from pipeline 2) comes from the state owner's information signal from the datapath in pipeline 2.

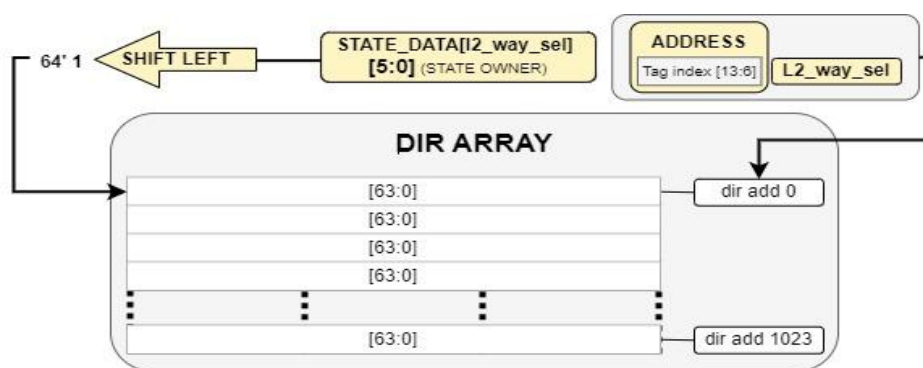


Figure 3.23 L2 directory array

State Array: This is used to store the information of each cache line of L2. It is 8x66 bits in size and contains 66 bits of data in each address, the breakdown of those 66 bits is represented below.

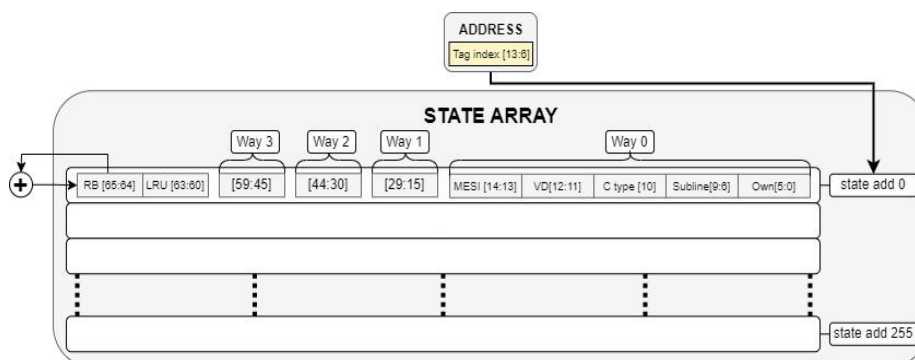


Figure 3.24 L2 state array

In the datapath of the pipe1 the eviction is being calculated, below mention is the L2 eviction algorithm.

Now, In the eviction algorithm, every VD bit is checked in ascending order and if all the valid and dirty bits of each way are high, then the eviction will be decided on the basis of round-robin bits.

- Let's understand with the example case:
- Assuming all the valid and dirty is high.
- Next, the algorithm will check the value in the RB bits.
- Suppose $RB=0$, then the algorithm will check for the lsu bits for each way.
- If way0 is not the least recently used data then, it will evict:
- Else way1 will be checked.
- This will continue till it will find the not least recently used way.

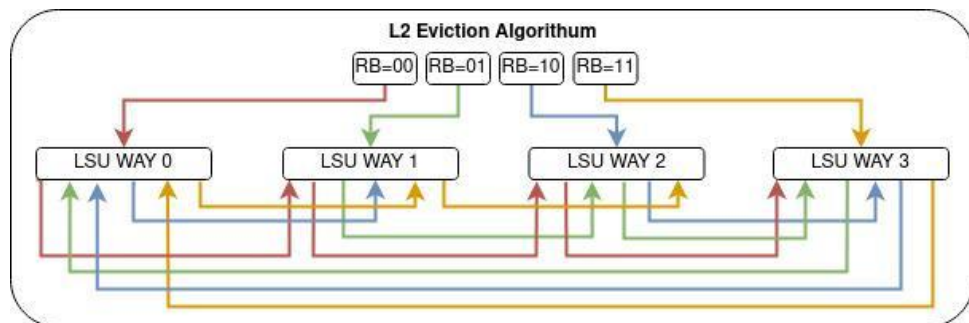


Figure 3.25 L2 cache replacement/eviction algorithm

Tag array: Tag memory is 104x8 bits, so each set is 104 bits; there are 256 sets containing 4 blocks of 26 bits each due to 4 way set associative cache (1024 tags in total).

In the address which may be coming from MSHR or directly from the message 26 bits [39:14] contain this tag information and 8 bits [13:6] contain the tag's address in the tag memory.

In stage 2 of both pipelines, tag hit decomposition occurs as:

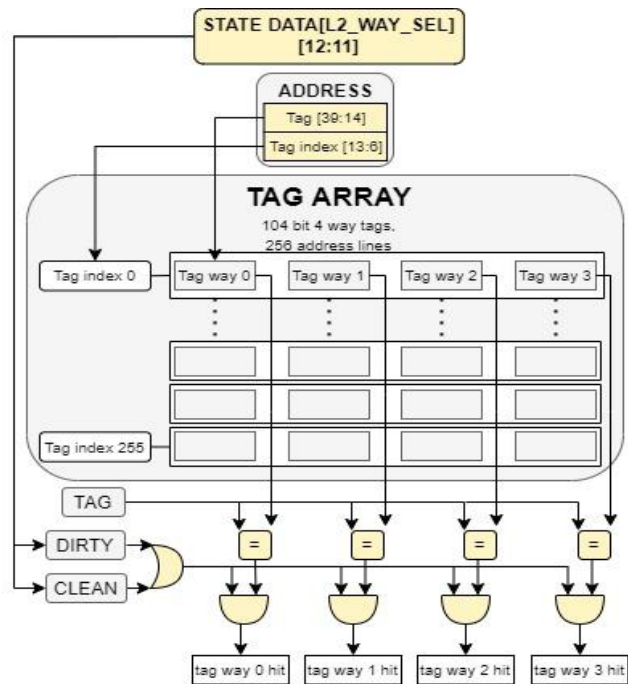


Figure 3.26 L2 tag array and tag hit

Data Array: The data array is the actual cache memory that stores the data.

It is 144x12 bit in size containing 1024 cache lines/blocks and 256 sets. Since 12 bits of the address are there, every 144 bits even in the cache line is accessible.

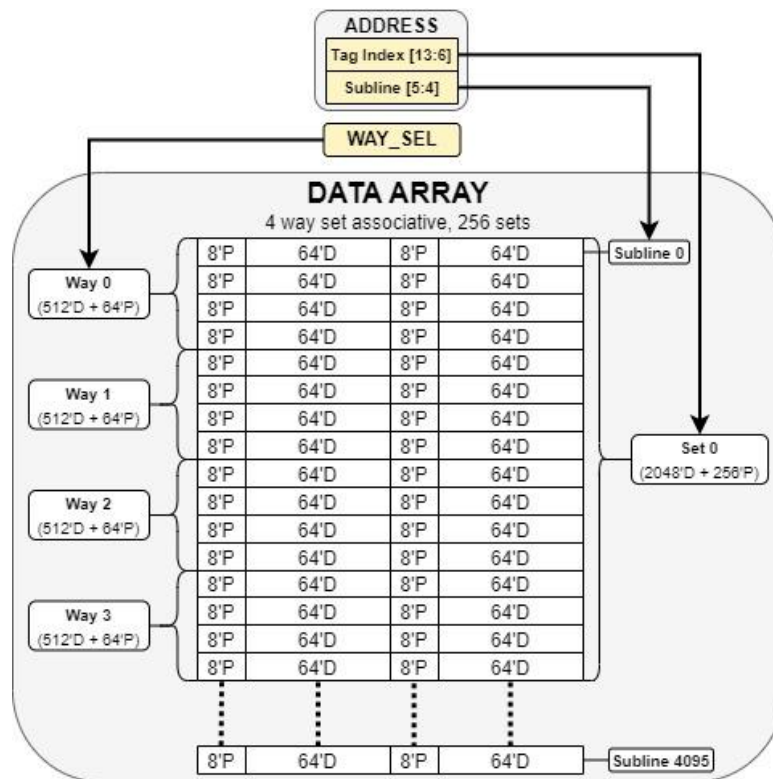


Figure 3.27 l2 data array

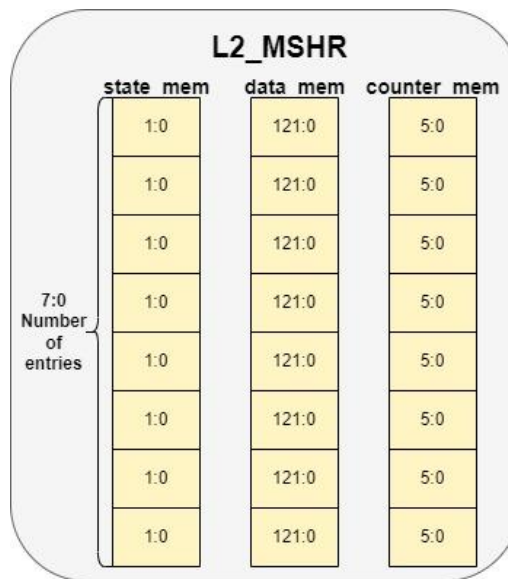
MSHR

Figure 3.28 L2 mshr

MSHR is a miss status holding register, it acts as a buffer for the outstanding cache misses and keeps track of the incoming data cache miss requests. So it performs as an agent between a cache miss and the miss output request transaction. It also keeps track of repeated input cache miss requests for the same data address, preventing repeated cache miss requests for the same addresses.

3.1.12. NOC Router: The main features of the openpiton+ariane NoC architecture are listed below:

The system is made up of several tiles, which are discrete processing components or functional units. Typically, an instance of the ariane processing core and all of its supporting parts are included in each tile.

Architecture Overview: To make sure that everything runs smoothly without any traffic jams, each network-on-chip (NoC) has a specific priority assigned to it.

NoC3 has the highest priority, followed by NoC2, and finally, NoC1, which has the lowest priority. This means that NoC3 will always be given first preference and won't be blocked by lower-priority NoCs. Additionally, all the hardware components are designed in a way that high-priority data can be processed independently without being held up by lower priority traffic.

Each NoC is responsible for handling a specific set of Coherence Operations. These operations are distributed in a way that guarantees deadlock-free operation of the NoC Routers, ensuring smooth and efficient data flow [5].

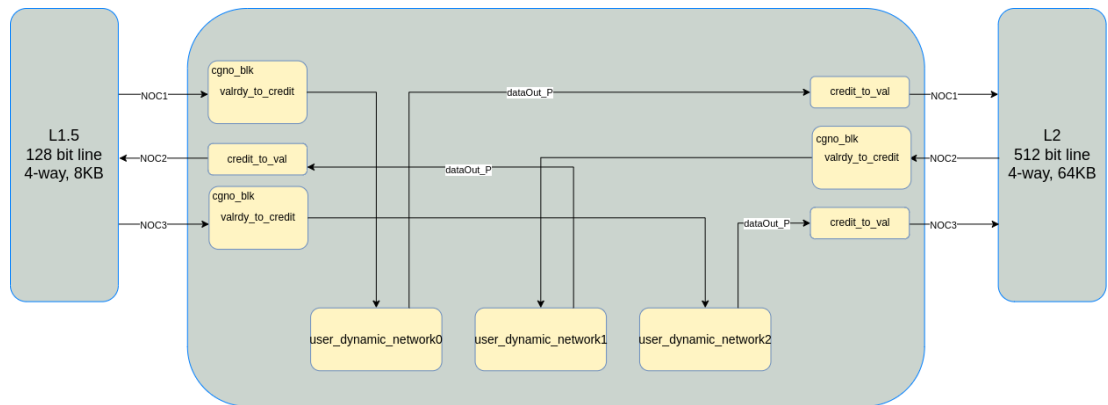


Figure 3.29 NOC Router Overview

NoC1 messages are created when the (L1.5) private cache asks the (L2) shared cache for data.

NoC2 messages are started by the (L2) and can be directed to either the (L1.5) or the memory controller.

NoC3 messages are responses from either the (L1.5) or the memory controller back to the (L2).

3.1.13. Chipset/Chipbridge: Within a single openpiton chip, the communication between tiles occurs through three P-Mesh networks located on the chip, which are known as NoC Routers. The routers are arranged in a flexible 2D mesh structure by default, enabling scalability. The address space of the routers supports the expansion of up to 256 tiles in each dimension.

The tile array is linked to external logic, such as a chipset implemented on an FPGA, through a connection interface known as the chip bridge. The chip bridge extends the functionality of the three NoC routers beyond the chip boundaries, multiplexing their communication over a single link.

When a memory request in the chip encounters a cache miss, the L1.5 cache plays a crucial role. It translates and forwards the request to the L2 cache through the NoC channels.

In general, the L1.5 cache initiates requests using NoC1, receives data via NoC2, and performs writebacks of modified cache lines using NoC3.

3.1.14. On-Chip Network: The primary role of the NoC Routers is to transmit appropriate flits based on their input. Within the openpiton framework, NoC1 messages are generated when the private cache (L1.5) requests data from the shared cache (L2). The NoC consists of two components: a buffer and an encoder.

Buffer: The NoC1 buffer stores data before it is sent to the encoder. Its purpose is to queue instructions and collect the complete message prior to transmission through the pipeline. This step is compulsory because requests in openpiton are transmitted as "flits," and since the largest size of a flit is 64 bytes. Hence, the buffer is used to gather all the flits comprising a message.

Encoder: The buffer consists of two separate parallel buffers: one "message header" and the second "message data". The design simplifies the logic by dividing the control path from the data path. The message header can be fetched in an initial stage, while the message data can be fetched at a later without affecting the processing of message headers.

3.1.15. NOC Router Waveform Debugging:

NOCI: In the openpiton system, NoC1 messages originate from requests made by the private cache (L1.5) to the shared cache (L2).

NoC1 determines the source of the flits based on corresponding flags that are set to a high state depending on the source transmitting the flits. By default, NoC1 selects the L1.5 cache as the source.

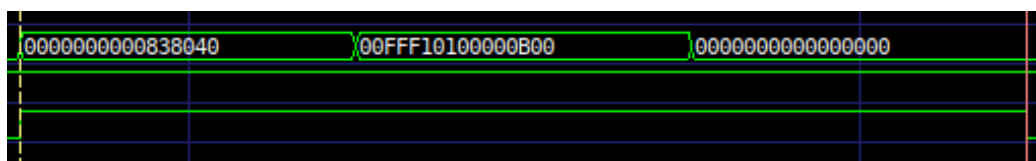


Figure 3.30 NOC 1 Wavefor

The above Figure 3.47 depicts a packet originating from L1.5 through NOC1. As described in the Packet Header Format for Coherence Requests table, it outlines the structure of a packet. Similarly, in this screenshot, we can observe that the packet is divided into three flits. By comparing this data manually with the table, we can determine the information it contains. Let's take a closer look.

Flit 1:

The above picture depicts the internal details of the flit. Where the first 8 bit shows the option 1 which is reserved. The next 8 bit depicts the MSHR/TAG. The next 8 bits MSG TYPE which if we convert it into decimal equal to 14 and which is the non-cacheable load request the payload length is 2 and FBITS is 0 which shows the request for the processor.

Flit 2:

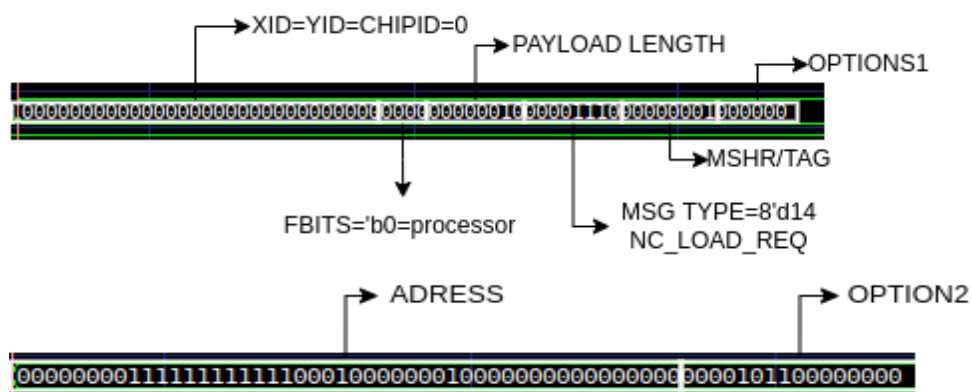


Figure 3.31 NOC Flit Overview

Now in the above picture the address depicts the physical address from the request side. But OPTION2 is subdivided into more fields like Sub-cache-line, Icache bit, and data size. Now out of 16 bits 8-10 3 bits of OPTION2 represent data size as shown in the option fields for requests and responses table. The 1 bit represents for Icache bit is it from Icache or dcache. And the last 4 bits show the SUBCACHELINE BIT VECTOR

Flit 3:

In the packet diagram the 3rd flit is 0 all time. OPTION 3 is 30 bit which is reserved. Flit 3 holds the information of the source. That is why SRCX and SRCY both equal zero which represents the tile zero.

NOC3: The messages sent via NoC3 are responses from the private cache (L1.5) to the shared cache (L2).

Unlike NoC1, NoC3 has a single source for its messages. Typically, a message can be transmitted in a single flit if there is no additional data or load accompanying it. If there is extra data or load to be transmitted with the message, it usually requires a total of three flits to transmit the complete message

3.2. Details of Software Implementation

3.2.1. Setting up the environment: The setup for running simulations with OpenPiton contains a script, `piton/piton_settings.bash`, that does most of the work [6]. First start with cloning and install riscv-toolchain in `/home/$USER` using these commands:

```
git clone --recursive
https://github.com/riscv/riscv-gnu-toolchain
cd /home/$USER
cd riscv-gnu-toolchain
./configure --prefix=/home/$USER/tools/riscv_install >
/dev/null
sudo bash
make
```

3.2.2. Simulation: To run a simulation with openpiton will require two components: a simulation model and a test. The sims tool is used to build models, run tests, and run regressions. It generates sims outputs log files, temporary

configuration files, and results files to the current working directory by default, hence it is needed to call sims from the build directory to keep all files that are generated in simulation process in one place.

The assembly tests are provided in directory `$PITON_ROOT/piton/verif/diag/assembly`. And all the C tests are located in `$PITON_ROOT/piton/verif/diag/c`. In addition, there are assembly files associated with each C test in `$PITON_ROOT/piton/verif/diag/assembly/c`. The assembly file has directives which instructs the assembler to invoke the compiler. The assembly directives point to the corresponding C files associated with this test. And then specify the associated assembly file the same way assembly tests are specified in order to run the C test.

Like assembly tests, there are many other arguments available when running C tests enabling for more complex simulations. These arguments are applicable to both assembly and C tests. C tests are run by referencing their associated assembly file which includes directives to the assembler to invoke the C compiler.

Running the assembler generates different files in the current working directory. A few are worth pointing out:

- `diag.exe` - an ELF formatted binary of the test
- `mem.image` - a memory image with the test code and associated infrastructure (boot code, interrupt handlers, etc.) embedded at correct addresses
- `midas.log` - a log of the assembler run
- `symbol.tbl` - the symbol table for the test

The Verilog simulation command line is built using the configuration files and different sims command line options. For assembly and C tests, the assembly

tests are passed to the Verilog simulator, while the test is actually read into the simulator from the **mem.image** file generated by the assembler. For unit tests, the source and sink memory files to be used are passed directly from the **sims** command line to the simulator command line through the **-sim run args=OPTION** argument. In the end, the command executes the simulation, logging the output to **sim.log**.

For the current version of multicore SoC the following command with the **vcd** flags is passed.

```
sims -sys=manycore -vlt_run -x_tiles=1 -y_tiles=1
rv64ui-p-addi.S -ariane -precompiled
-vlt_build_args=--trace -vlt_build_args=-CFLAGS
-vlt_build_args=-DVERILATOR_VCD
```

Aws-Fpga AFI Generation Flow: AFI (Amazon FPGA Image) is a bitstream file used to program FPGA. Following steps are needed to be run to generate AFI [7].

- **Step 1: Build**

Start the `aws_build_dcp_from_custom_logic.sh` build script in the `$CL_DIR/build/scripts` directory.

```
$ cd build/scripts
```

Now build the script by using this command,

```
$ ./aws_build_dcp_from_custom_logic.sh -clock_recipe_a A2
-strategy CONGESTION -foreground
```

The outputs of the build script are:

- `$CL_DIR/build/checkpoints/*:`

Various checkpoints generated during the build process.

- `$CL_DIR/build/to_aws/SH_CL_routed.dcp:`

Encrypted placed-and-routed design checkpoint for AWS ingestion.

- `$CL_DIR/build/reports/*:`

Various build reports (generally, `check_timing/report_timing`).

- `$CL_DIR/build/src_post_encryption/*:`

Encrypted developer source.

- `$CL_DIR/build/constraints/*:`

Implementation constraints.

- **Step 2: Sourcing HDK & SDK files**

First, Move to the root directory of the repository (`aws-fpga`) before running the next script. Commands to source the hdk and sdk setup are shown in the `aws-fpga platform[15]`.

- **Step 3: Environment Setup of your CL Directory**

The developer design repository's vivado design folder should be the target of the `CL_DIR` environment variable. Commands to set up the environment for cl directory are shown in the `aws-fpga platform[15]`.

- **Step 4: Setup CLI and Create S3 Bucket**

For the AFI generation, the developer must set up an S3 bucket. A tar file and logs produced by the AFI creation service will be stored in the bucket.

First step is to set your AWS-FPGA credentials by running the command `"aws configure"`. And then set all the credentials (i.e. AWS access key id, secret key, region name, output format).

- **Step 5: Create S3 bucket (folder) on aws server**

The AWS scripts will use this S3 bucket to upload your DCP to AWS in order to create an AFI that will be compressed as a tar file.

Start by creating a bucket:

```
$ aws s3 mb s3://piton --region us-east-1 # Make a S3 bucket.
```

Pick a memorable bucket name.

```
$ aws s3 mb s3://piton/piton_dcp # Make a folder for the files in your tarball.
```

- **Step 6: Copy DCP to S3 bucket**

After create a bucket, then copy it to S3

```
$ aws s3 cp $CL_DIR/build/checkpoints/to_aws/*.Developer_CL.tar  
s3://piton/piton_dcp/
```

NOTICE: The trailing '/' is necessary after dcp-folder-name.

- **Step 7: Create a folder for your log files**

Now, Create a folder to save your logs by following the steps below

```
$ aws s3 mb s3://piton/piton-logs # Establish a folder for your logs.  
  
$ touch LOGS_FILES_GO_HERE.txt # Establish a temp file  
  
$ aws s3 cp LOGS_FILES_GO_HERE.txt s3://piton/piton-logs/
```

- **Step 8: Check Policy**

Check your policy (do you have correct rights to create AFI) [optional]

```
$ check_s3_bucket_policy.py --dcp-bucket piton --dcp-key  
piton_dcp/<tar-file-name> --logs-bucket piton --logs-key  
piton-logs
```

- **Step 9: Start AFI Creation**

Create the Amazon FPGA image (AFI) after your policy has passed the checks.

```
$ aws ec2 create-fpga-image --name piton-afi --description  
afi_for_multi_core_soc_for_smp_linux --input-storage-location  
Bucket=piton_dcp,Key=<path-to-tarball> --logs-storage-location  
Bucket=piton,Key=piton-logs
```

The output of this command includes two IDs that match your AFI:

- ***FPGA Image Identifier or AFI ID:*** The primary ID used to manage your AFI using the AWS EC2 CLI commands and AWS SDK APIs is the FPGA Image Identifier, also known as the AFI ID. Given that this ID is regional, if an AFI is duplicated across numerous areas, each region will give it a unique AFI ID. AFI ID `afi-0f4ff82e29c6494f9` is an example.
- ***Global FPGA Image Identifier or AGFI ID:*** Global ID, also known as an AGFI ID or FPGA Image Identifier, is how an AGFI is identified within an F1 instance and is used to identify it. An AFI can be loaded or unloaded from an FPGA slot using, for example, the AGFI ID. As a result of the AGFI IDs being global (by design), you can replicate a combination of AFI/AMI to several other areas, and they will all work without any additional setup. The AGFI ID `01e33810ff4c9d23c` serves as an illustrative example.

- **Step 9: Provide AFI_ID**

Then check if the AFI generation is done. When `create-fpga-image` returns its result, you must provide the following information:

```
$ aws ec2 describe-fpga-images --fpga-image-ids
afi-0f4ff82e29c6494f9
```

- **Step 10: Load the AFI**

Using the FPGA Management tools in your F1 instance, you can now load your AFI onto an FPGA in a specific slot. Make sure to unload any AFI that was loaded into your slot previously. You can load the AFI by using the command `"fpga-clear-local-image -S 0"` with `sudo`.

If the state of the `fpga-describe-local-image` API function is "Busy," the FPGA is still carrying out the previous operation in the background. As

previously mentioned, wait until the status says "Cleared."

Attempting to load your AFI into FPGA slot 0 now:

```
$ sudo fpga-load-local-image -S 0 -I agfi-01e33810ff4c9d23c
```

REMINDER: The FPGA Management tools use the AGFI ID rather than the AFI ID.

CHAPTER # 04

PROJECT CHARACTERIZATION

4.1. Results

This section reflects what the project has achieved and serve as a representation of the outcomes of the processes involved, these processes are:

4.1.1. Simulations: For simulating the hardware, there are multiple types of pre-built test suite available, all the tests are passed and running to their full. Below attached are the outputs of them.

4.1.2. QuadCore Simulation Testing: A bare Metal Application of Hello World Tests results are showing below:

```
sims -sys=manycore -vlt_run -x_tiles=2 -y_tiles=2
hello_world.c -ariane -rtl_timeout 10000000.
```

Result

```
Hello world, this is hart 0 of 4 harts!
Hello world, this is hart 1 of 4 harts!
Hello world, this is hart 2 of 4 harts!
Hello world, this is hart 3 of 4 harts!
```

Figure 4.1 Thread Running On Each Core

The test was expected to print these statements on the fake_UART log file.

GTKWave .VCD waveform

With the monitor's output, there is another way which is useful when we want to debug the running program on the signal level. This is called simulation waveform debugging.

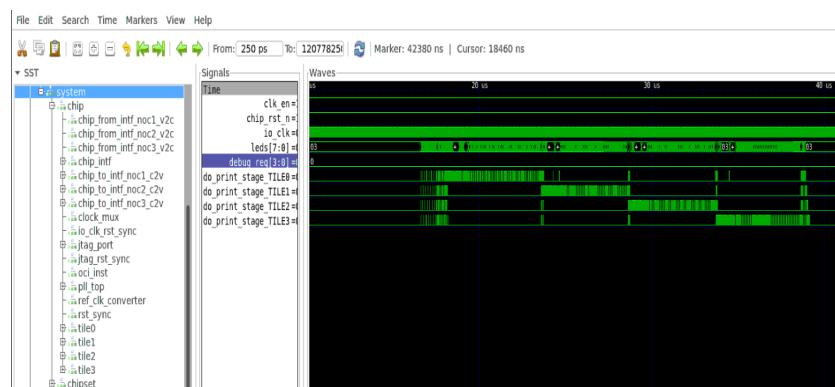


Figure 4.2 GTKWave Form

4.1.3. Regression Testing: OpenPiton provides grouping options for the tests, which means the multiple tests of the same or different type can be grouped together and run on a single simulation model as shown:

Status: ariane_tile1_benchmarks		ALL	
PASS:	8	8	
FAIL:	0	0	
Diag Problem:	0	0	
License Problem:	0	0	
MaxCycles Hit:	0	0	
Socket Problem:	0	0	
Timeout:	0	0	
LessThreads:	0	0	
Simics Problem:	0	0	
Performance:	0	0	
Killed By Job Q:	0	0	
Unknown:	0	0	
UnFinished:	0	0	
flexlm error:	0	0	
Diag Count:	8	8	
Cycles/Sec:	-NA-	-NA-	
K Cycles:	0	0	
#Diags Used:	0	0	
REGRESSION PASSED			

Figure 4.3 Benchmarks Regression Testing Results

Scripting gathers all the test results, compiles them and then displays the final results of the regressions [5].

4.1.4. Outcome Of AFI(AWS FPGA Image):

```
[rafia@ip-172-33-7-206 aws-fpga]$ aws ec2 describe-fpga-images --fpga-image-ids afi-061a52a3f87a4676b
{
  "FpgaImages": [
    {
      "UpdateTime": "2023-01-01T16:55:08.000Z",
      "Name": "int_test",
      "Tags": [],
      "PciId": {
        "SubsystemVendorId": "0xfedc",
        "VendorId": "0x1d0f",
        "DeviceId": "0xf001",
        "SubsystemId": "0x1d51"
      },
      "FpgaImageGlobalId": "agfi-0d5a30d3f99a22172",
      "Public": false,
      "State": {
        "Code": "available"
      },
      "ShellVersion": "0x04261818",
      "OwnerId": "040557801715",
      "FpgaImageId": "afi-061a52a3f87a4676b",
      "CreateTime": "2023-01-01T15:52:23.000Z",
      "Description": "testing bare-metal test"
    }
  ]
}
```

Figure 4.4 Available Global ID Of AFI

4.1.5. Bare Metal Application On AWS FPGA: The goal of bare metal testing is to measure the raw performance and capabilities of the hardware, without any interference or overhead from the operating system or other software.

4.1.6. Software Compilation And Runtime Driver

`cd $CL_DIR/software/src && make` # This will compile the program for DMA and UART, reset

Runtime Driver of UART which helps to connect sudo terminal
`sudo ./UART &` Run UART program # This will create a pseudo-terminal and tell the location of the corresponding file.

```
[muheet@ip-172-33-7-206 src]$ sudo ./uart &
[2] 13854
[muheet@ip-172-33-7-206 src]$ terminal is open at /dev/pts/4
sudo tio /dev/pts/4
[22:13:22.789] tio v2.2
[22:13:22.789] Press ctrl-t q to quit
[22:13:22.789] Connected
```

Figure 4.5 Output Of Runtime Driver UART

4.1.7. Result Of Bare Metal Application On AWS FPGA: After writing Instruction in memory through DMA `sudo ./dma_os(for .bin)` `sudo ./DDR_Loader(for .hex)`. Reset FPGA through `sudo ./fpga-reset` the process will start working and printing UART data in pseudo-terminal

```
[muheet@ip-172-33-7-206 src]$ sudo ./uart &
[2] 13854
[muheet@ip-172-33-7-206 src]$ terminal is open at /dev/pts/4
sudo tio /dev/pts/4
[22:13:22.789] tio v2.2
[22:13:22.789] Press ctrl-t q to quit
[22:13:22.789] Connected

-----
--      OpenPiton+Ariane Platform      --
-----
OpenPiton Version: b'c40d92a7'
Ariane Version:    b'cbbcf356'

FPGA Board:       f1
Build Date:       Nov 06 2022 20:13:08

#X-Tiles:         2
#Y-Tiles:         2
#Cores:           4
Core Freq:        62 Mhz
Network:          2dmesh.config
DRAM Size:        4096 MB

L1I Size / Assoc: 16 kB / 4
L1D Size / Assoc: 8 kB / 4
L1S Size / Assoc: 8 kB / 4
L2 Size / Assoc:  64 kB / 4
-----

sd initialized!
```

Figure 4.6 Output Of Bare Metal Application

4.1.8. Booting Linux: Same as the previous step we will write the linux image on the DDR through the DMA_Loader runtime loader.

Figure 4.7 Booting Linux Output

4.2. Analysis

The project has multiple areas where one can get the opportunity to learn and get an idea about the functionality, internal architecture, and the relationship between software and hardware components.

Last, there is also a network invalidation protocol which comes into account when a device or in this project a core needs to send the invalidation to another core about the particular data. In the openpiton Directory based is used However there can be a snooping protocol that generally gets implemented where there is less number of cores.

4.2.1. Power Analysis: Power analysis from the implemented netlist.

Activity derived from constraints files

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 2.427 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 29.3°C
 Thermal Margin: 55.7°C (30.5 W)
 Effective θ_{JA} : 1.8°C/W
 Power supplied to off-chip devices: 0.31 W
 Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

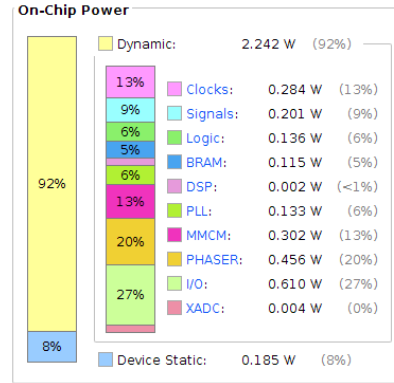


Figure 4.8 Power Analysis

4.2.2. Utilization Analysis of Post Synthesis: These are the utilization reports which show the areas and the resources in the FPGA that would be used when wanting to port the hardware.

Table 1 Utilization Analysis Of Post Synthesis

Graph Table			
Resource	Estimation	Available	Utilization %
LUT	143562	203800	70.44
LUTRAM	2545	64000	3.98
FF	95213	407600	23.36
BRAM	117	445	26.29
DSP	55	840	6.55
IO	50	500	10.00
BUFG	5	32	15.63

4.3. Conclusion

The output we have achieved is an actual System on Chip containing 4 cores which are emulated on field programmable gate arrays, In this project 4 cores are ported on AWS-FPGA and SMP linux as the OS running. Furthermore, the bitstream for Genesys2.

From the learning and the project representation perspective Zephyr operating system is also booted on the Litex wex_risc-v core on arty a7 FPGA.

4.4. Future Recommendations

After a comprehensive analysis, to enhance the performance and usability of such systems the key challenges which can be catered using OpenPiton's scalable framework are to investigate techniques to explore advanced caching mechanisms and memory management strategies to enhance data access and minimize latency in SMP Linux environments. An approach to use dynamic power management techniques to have optimal performance having minimized power consumption and focus on developing real-time scheduling policies and exploring hardware-based mechanisms that guarantee predictable execution times and response latencies. Promote collaboration and active engagement with the open-source community to foster continuous improvements and knowledge sharing. Encourage contributions, conduct workshops, and disseminate research findings to the broader community to stimulate further advancements in multicore SoCs and SMP Linux using the OpenPiton framework. By addressing these challenges and exploring opportunities for improvement, researchers and developers can unlock the more potential of these systems, contributing to a more efficient, scalable, and reliable computing ecosystem which can lead towards fabrication of the project.

REFERENCES

- [1] The NOVA Project <https://github.com/The-Nova-Project/Nova1/tree/main>.
- [2] David A. Patterson and John L. Hennessy, Computer Organization and Design. Morgan Kaufmann Publishers.
- [3] openpiton: An open source Manycore Research Framework, 2016. <http://parallel.princeton.edu/papers/openpiton-asplos16.pdf>
- [4] Piton: A Manycore Processor For Multinant Clouds. 2017. <http://www.parallel.princeton.edu/papers/piton-micro-mag-17.pdf>
- [5] openpiton Microarchitecture Specification, Wentzlaff Parallel Research Group, Princeton University, Feb, 04, 2016. http://parallel.princeton.edu/openpiton/docs/micro_arch.pdf
- [6] openpiton Simulation Manual, Wentzlaff Parallel Research Group, Princeton University, {version 2.3} http://parallel.princeton.edu/openpiton/docs/sim_man.pdf
- [7] openpiton FPGA Prototype Manual, Wentzlaff Parallel Research Group, Princeton University [Version 3.0] http://parallel.princeton.edu/openpiton/docs/fpga_man.pdf
- [8] openpiton Synthesis and Back-end Manual, Wentzlaff Parallel Research Group, Princeton University, {version 1.2} http://parallel.princeton.edu/openpiton/docs/synbck_man.pdf
- [9] operating System and System Software Tutorial, openpiton with RISC-V Cores, A Hands-On Tutorial with the opensource Manycore Processor. Micro 2019 Columbus. http://parallel.princeton.edu/openpiton/tutorial_slides/micro19/6-openpitonariane-os.pdf
- [10] <https://github.com/PrincetonUniversity/openpiton>
- [11] “RISC-V International.” <https://riscv.org/> (accessed Jun. 22, 2022).
- [12] Ariane, https://cva6.readthedocs.io/en/latest/03_cva6_design/index.html
- [13] “IEEEExploreFull-TextPDF:”<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9524573> (accessed Jun. 22, 2022).
- [14] J. Balkind et al., “openpiton+ariane: The First opensource, SMP linux-booting RISC-V System Scaling From One to Many Cores,” Third Work. Comput. Archit. Res. with RISC-V,CARRV,2019,[Online].Available: https://parallel.princeton.edu/papers/balkind_carrv2019.pdf.
- [15] <https://github.com/aws/aws-fpga/blob/master/hdk/README.md>

APPENDIX A: Complex Engineering Problem

Appendix A1: Range of Resources

Diversified resources are crucial for the development of a multicore System-on-Chip. It involves hardware design, firmware development, low-level software integration, and operating system support. This project demands seamless integration between hardware and software components. Achieving high performance and power efficiency in a multicore SoC with expertise in various optimization strategies to enhance the overall system performance.

Table 2 Range of Resources

Hardware Resource			
S. No.	Discipline	Resource	Description
1	RTL Design	Hardware Design Engineer	Helped us to verify our SoC
2	Verification	Verification Engineer	Helped us to test each module
Software Resource			
S. No.	Discipline	Resource	Description
1	Linux Boot	Software Engineer	Used to run multiple applications on it. (i.e. games)
Equipment			
S. No.	Discipline	Resource	Description
1.	RTL Emulation	FPGA	Used to Emulate our hardware
Reference Literature			
S. No.	Discipline	Resource	Description
1	Computer Architecture	Articles/Books/Research Papers	Literature that provides sufficient information necessary to understand the working of the human brain and emerging signals.

Appendix A2: Innovation

The innovation that our project brings is the development of custom hardware and deployment on both Virtual FPGA (AWS-FPGA) and Physical FPGA (Genesys 2), then also booting SMP linux on it. The ISA we used is completely opensource and can be used for in house development of various hardware. Any kind of custom application specific processor can be designed for commercial or other purposes and this project serves an example.

Appendix A3: Level of Interaction

The diversified issues are found when the multicore processor system along with cache coherence and the interactions with the peripherals and system software running on the hardware are all combined. The problems emerged were porting the whole system on one FPGA, the SoC design exceeded the size of actual hardware with complete multicore specifications and only one core could be ported on Genesys 2.

Appendix A4: Consequences to Society and Environment

This empowerment of students and researchers with cutting-edge expertise prepares them for the demands of the tech industry, driving innovation and technological advancement. Economically, the project enables cost-effective multicore solutions and reduces dependency on limited chip suppliers, benefiting various industries. In terms of security, in-house chip design ensures greater control over critical systems, particularly in defense applications, reducing potential risks of backdoors and security breaches. The project's overall impacts contribute to a more technologically empowered and secure society.

Appendix A5: Familiarity

In this project, multicore SoC for SMP linux using openpiton+ariane, valuable familiarity was gained with the design, implementation, and practical applications of multicore architectures. The project involved configuring the simulation environment, customizing the ariane core, and booting SMP linux on the multicore processor.